

UNIT V:

- ❑ **Transaction Concept:** Transaction State, ACID properties, Concurrent Executions, Serializability, Recoverability, Implementation of Isolation, Testing for Serializability, lock based, time stamp based, optimistic, concurrency protocols, Deadlocks, Failure Classification, Storage, Recovery and Atomicity, Recovery algorithm.
- ❑ **Introduction to Indexing Techniques:** B+ Trees, operations on B+Trees, Hash Based Indexing

UNIT V:

- **Transaction Concept:** Transaction State, ACID properties, Concurrent Executions, Serializability, Recoverability, Implementation of Isolation, Testing for Serializability, lock based, time stamp based, optimistic, concurrency protocols, Deadlocks, Failure Classification, Storage, Recovery and Atomicity, Recovery algorithm.
- Introduction to Indexing Techniques: B+ Trees, operations on B+Trees, Hash Based Indexing

Transaction

- ❑ A transaction is an **event which occurs on the database**.
- ❑ Database **Transaction is an atomic unit** that contains one or more SQL statements.
- ❑ Generally a transaction **reads a value** from the database or **writes a value** to the database.
- ❑ **It is a series of operations** that performs as a single unit of work against a database.
- ❑ It has a beginning and an end to **specify its boundary**

Transaction

- E.g., transaction to **transfer \$50 from account A to account B**:
 - 1. **read(A)**
 - 2. **$A := A - 50$**
 - 3. **write(A)**
 - 4. **read(B)**
 - 5. **$B := B + 50$**
 - 6. **write(B)**

Transaction

Transaction Begins

```
UPDATE savings_accounts  
  SET balance = balance - 500  
  WHERE account = 3209;
```

Decrement Savings Account

```
UPDATE checking_accounts  
  SET balance = balance + 500  
  WHERE account = 3208;
```

Increment Checking Account

```
INSERT INTO journal VALUES  
  (journal_seq.NEXTVAL, '1B'  
   3209, 3208, 500);
```

Record in Transaction Journal

```
COMMIT WORK;
```

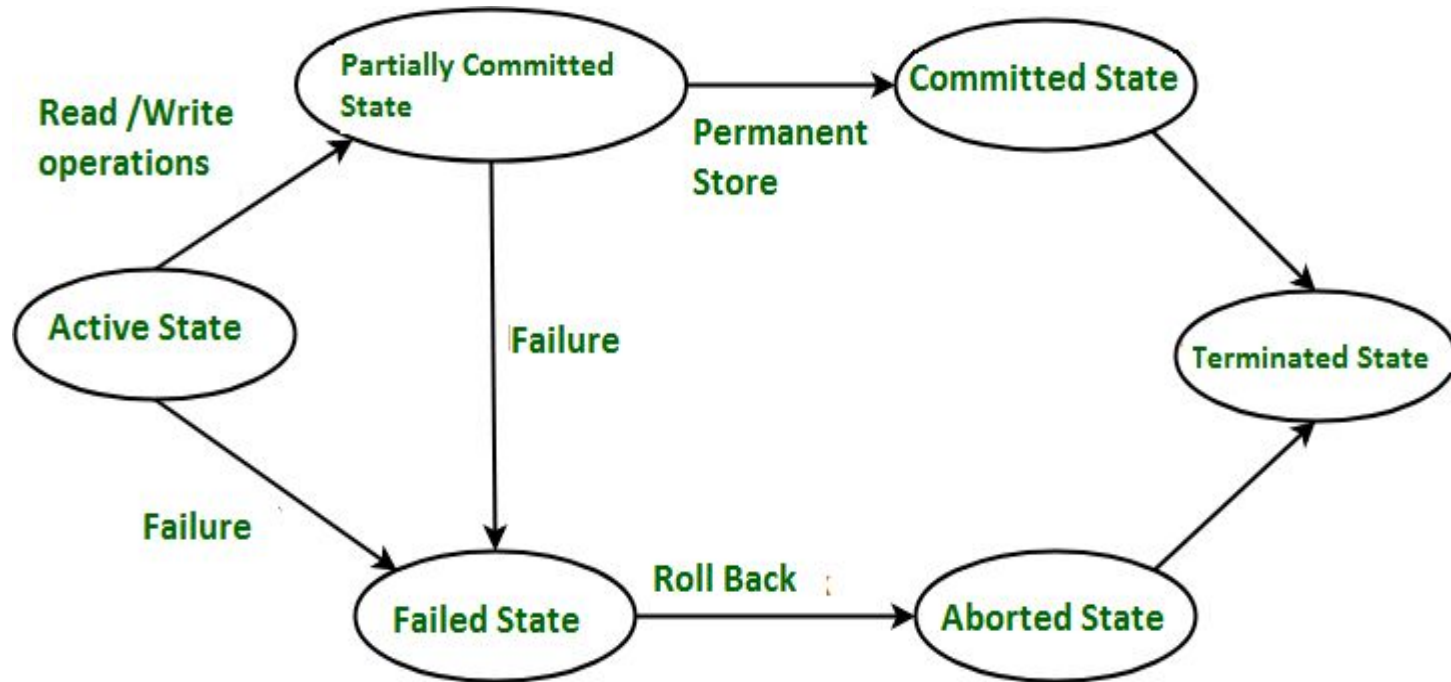
End Transaction

Transaction Ends

UNIT V:

- Transaction Concept: **Transaction State**, ACID properties, Concurrent Executions, Serializability, Recoverability, Implementation of Isolation, Testing for Serializability, lock based, time stamp based, optimistic, concurrency protocols, Deadlocks, Failure Classification, Storage, Recovery and Atomicity, Recovery algorithm.
- Introduction to Indexing Techniques: B+ Trees, operations on B+Trees, Hash Based Indexing

Transaction State



Transaction States in DBMS

Transaction State

□ Active State

- **Description:** The initial state where the transaction's operations (like reading and writing data) are being executed.
- **Transition:** Moves to the next state if operations complete successfully, or to the Failed state if an error occurs.

□ Partially Committed State

- **Description:** The transaction has completed its final operation but has not yet committed to the database.
- **Transition:** If the commit operation is successful, it moves to Committed. If an error occurs here, it goes to the Failed state.

Transaction State

□ Committed State

- **Description:** All of the transaction's operations have been successfully completed, and the changes are made permanently to the database.
- **Transition:** The transaction is now considered complete and moves to the Terminated state.

□ Failed State

- **Description:** An error is detected during the transaction's execution (e.g., power failure, hardware error), and the transaction cannot proceed further.
- **Transition:** Moves to the Aborted state for recovery.

Transaction State

□ Aborted State

- **Description:** The recovery manager has undone any changes made by the transaction to restore the database to its state before the transaction started.
- **Transition:** The transaction is terminated, moving to the Terminated state.

□ Terminated State

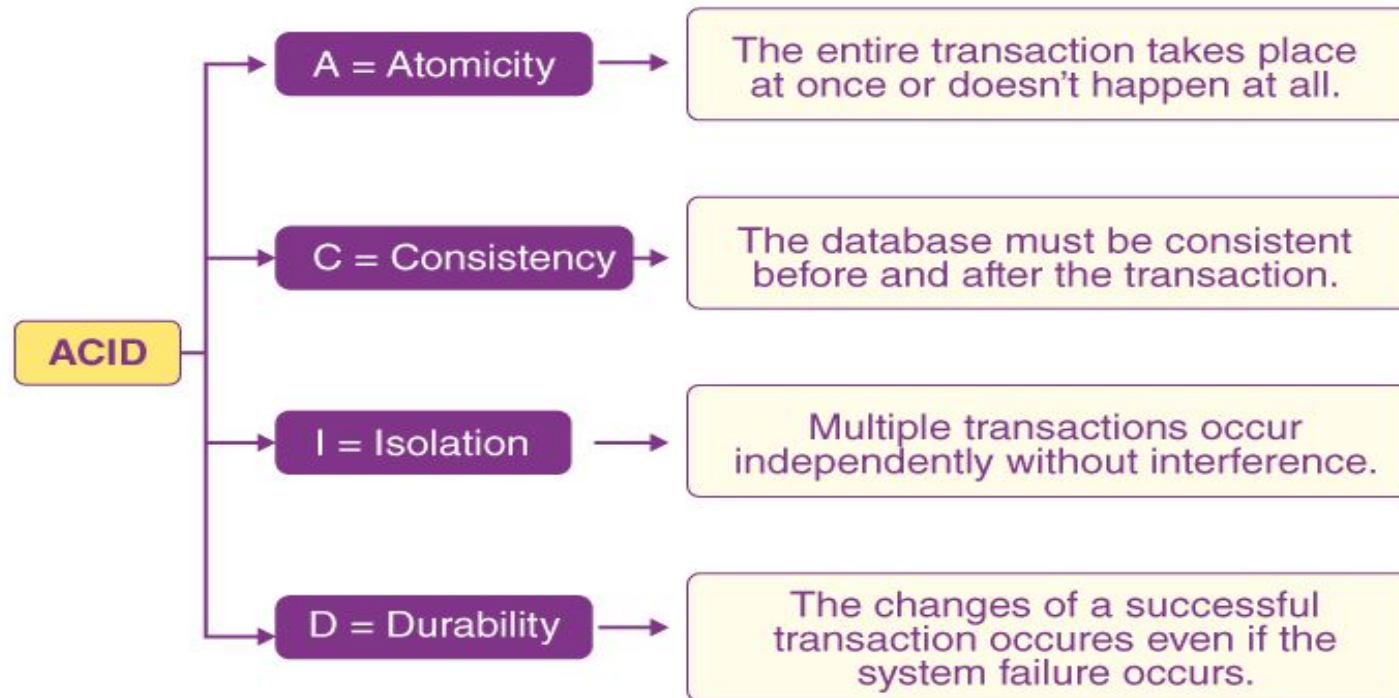
- **Description:** The final state of the transaction, reached after it has either been successfully committed or aborted.
- **Transition:** The transaction has ended and is removed from the system.

UNIT V:

- Transaction Concept: Transaction State, **ACID properties**, Concurrent Executions, Serializability, Recoverability, Implementation of Isolation, Testing for Serializability, lock based, time stamp based, optimistic, concurrency protocols, Deadlocks, Failure Classification, Storage, Recovery and Atomicity, Recovery algorithm.
- Introduction to Indexing Techniques: B+ Trees, operations on B+Trees, Hash Based Indexing

ACID properties

ACID Properties in DBMS



ACID properties

- ❑ Atomicity defines that **all operations** of the transactions are **either executed or none**.
- ❑ Atomicity is also known as '**All or Nothing**', it means that either perform the operations or not perform at all.
- ❑ It is maintained in the **presence of deadlocks, CPU failures**, disk failures, database and application software failures.

ACID properties

- ❑ Consistency defines that after the **transaction is finished**, the database must remain in a consistent state.
- ❑ It preserves consistency of the database.
- ❑ **If the transaction fails**, then the transaction will be **rolled back** and the database will be restored to a state consistent.
- ❑ **Consistency is said to be failed if atomicity fails**

ACID properties

- ❑ Isolation is the property which guarantees that the **execution of one transaction should not interfere with the execution of another transaction.**
- ❑ Isolation property **does not ensure the order** of transactions.
- ❑ **Isolation is needed** when there are concurrent transactions occurring at the same time.

ACID properties

- In database systems, **durability** is the ACID property which **guarantees that transactions that have committed will survive permanently**.
- For example, if a flight booking reports that a seat has successfully been booked, then the seat will remain booked even if the system crashes.

UNIT V:

- Transaction Concept: Transaction State, ACID properties, **Concurrent Executions**, Serializability, Recoverability, Implementation of Isolation, Testing for Serializability, lock based, time stamp based, optimistic, concurrency protocols, Deadlocks, Failure Classification, Storage, Recovery and Atomicity, Recovery algorithm.
- Introduction to Indexing Techniques: B+ Trees, operations on B+Trees, Hash Based Indexing

Concurrent Executions

- Concurrent execution in DBMS is the **simultaneous execution of multiple transactions on a single database, allowing multiple users or applications to access and modify data at the same time.**
- This technique improves system efficiency and throughput by allowing **operations to be interleaved, reducing wait times and increasing resource utilization.**
- However, it introduces **data inconsistency problems** and requires careful management through concurrency control techniques to maintain data integrity.

Concurrent Executions

- The main problems are:
 - Lost updates (WW Conflict)
 - Dirty Read or Temporary update problem (WR)
 - Unrepeatable read or incorrect analysis problem (RW)
 - Phantom Reads

Concurrent Executions: WW Conflict

- Lost Updates –(WW Conflict): The problem occurs when **two different database transactions perform the write operations on the same database items** in an interleaved manner.

Time	T _x	T _y
t ₁	READ (A)	—
t ₂	A = A - 50	—
t ₃	—	READ (A)
t ₄	—	A = A + 100
t ₅	—	—
t ₆	WRITE (A)	—
t ₇	—	WRITE (A)

LOST UPDATE PROBLEM

Concurrent Executions:WR Conflict

- Dirty Read Problems (W-R Conflict)
 - The dirty read problem occurs when one transaction **updates** an item of the database, and somehow the transaction fails, and before the data gets rollback, the updated database item is accessed by another transaction.
 - For example:
 - Consider two transactions Tx and Ty in the below diagram performing read/write operations on account A where the available balance in account A is \$300

Concurrent Executions: WR Conflict

Time	T_x	T_y
t_1	READ (A)	—
t_2	$A = A + 50$	—
t_3	WRITE (A)	—
t_4	—	READ (A)
t_5	SERVER DOWN ROLLBACK	—

DIRTY READ PROBLEM

Concurrent Executions: RW Conflict

- **Unrepeatable Read Problem(RW):** The unrepeatable problem occurs when **two or more read operations of the same transaction read different values of the same variable.**

Time	T _x	T _y
t ₁	READ (A)	—
t ₂	—	READ (A)
t ₃	—	A = A + 100
t ₄	—	WRITE (A)
t ₅	READ (A)	—

UNREPEATABLE READ PROBLEM

Concurrent Executions: Phantom Read

- **Phantom Read Problem:** The phantom read problem occurs when a transaction **reads a variable once but when it tries to read that same variable again, an error occurs** saying that the variable does not exist.

T1	T2
Read(X)	
	Read(X)
Delete(X)	
	Read(X)

UNIT V:

- Transaction Concept: Transaction State, ACID properties, Concurrent Executions, **Serializability**, Recoverability, Implementation of Isolation, **Testing for Serializability**, lock based, time stamp based, optimistic, concurrency protocols, Deadlocks, Failure Classification, Storage, Recovery and Atomicity, Recovery algorithm.
- Introduction to Indexing Techniques: B+ Trees, operations on B+Trees, Hash Based Indexing

Serializability, Testing for Serializability

- Types of Schedules
- Serializable schedule

Serializability, Testing for Serializability

- **Types of Schedules**
- **Serializable schedule**

Serializability, Testing for Serializability

- Schedule, as the name suggests, is a **process of lining the transactions** and executing them one by one.
- When there are multiple transactions that are running in a concurrent manner **Scheduling is brought into play.**
- Types of Schedules:
 - Serial Schedule
 - Non Serial Schedule

Serializability, Testing for Serializability

- Serial schedule is one in which **no transaction starts until a running transaction has ended.**

T1	T2
R(A)	
W(A)	
	R(A)
	W(A)

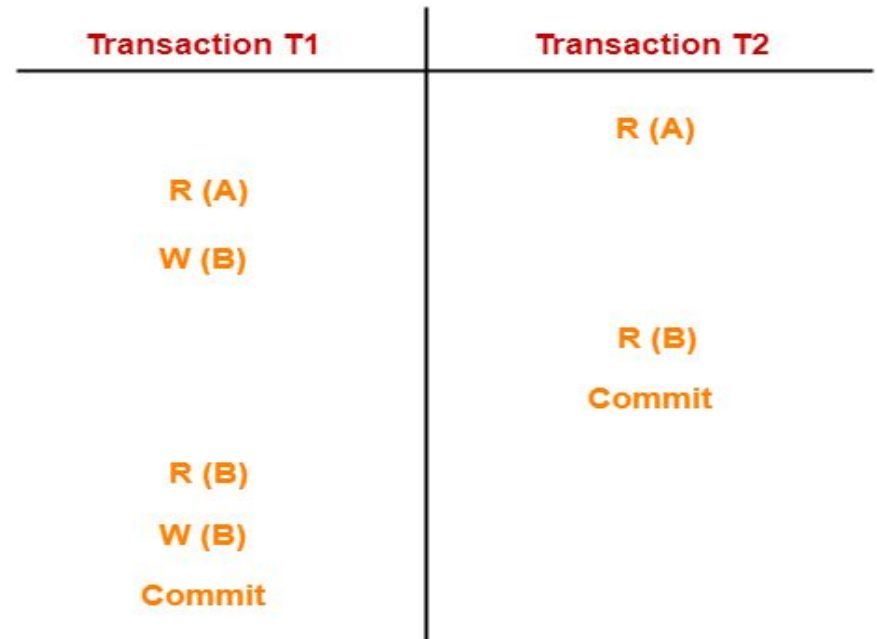
- Consider the schedule involving two transactions T_1 and T_2 .

Serializability, Testing for Serializability

- If it has **no interleaving of operations**, then there are the following two possible outcomes:
 - Execute all the operations of T1 which was followed by all the operations of T2.
 - Execute all the operations of T2 which was followed by all the operations of T1.
- **$T1 \sqsubseteq T2$ or $T2 \sqsubseteq T1$**
- **No. of schedules = $n!$**

Serializability, Testing for Serializability

- In non-serial schedules,
 - Multiple transactions execute concurrently.
 - Operations of all the transactions are inter leaved or mixed with each other.
 - Precedence graph will have loop or cycle in non-serial or parallel Schedule.

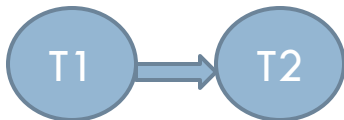


Serializability, Testing for Serializability

- Types of Schedules
- **Serializable schedule**

Serializability, Testing for Serializability

- Serializability is the ability of a schedule to become serializable.
- Precedence graph is used to denote the serializability of the schedule.
- Example: Consider a schedule 'S' with two transactions T1 & T2

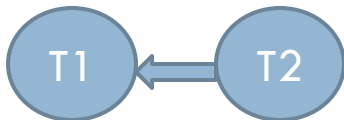


T1	T2
R(A)	
W(A)	
	R(A)
	W(A)

Serializability, Testing for Serializability

□ Example:

T1	T2
	R(A)
	W(A)
R(A)	
W(A)	



- As both are serial schedules, there is no need to go for serializability.

Serializability, Testing for Serializability

- Serializability of a schedule is needed for Parallel or non Serializable schedules.
- Serializability is the process of finding a serial schedule for the given parallel schedule.
- If there is cycle or loop in the given schedule then we can say that the given schedule is parallel.
- To verify whether the given schedule is serializable or not we use: Conflict Serializability, View Serializability.

Serializability, Testing for Serializability

- Example: Consider a schedule 'S' with two transactions T1 & T2

T1	T2
R(A)	
	R(A)
	W(A)
W(A)	

- T1 □ T2 □ T1



Serializability, Testing for Serializability

- For the given parallel schedule we should find a equivalent serial schedule.
- $T1 \sqsubseteq T2$ or $T2 \sqsubseteq T1$

T1	T2
R(A)	
W(A)	
	R(A)
	W(A)

T1	T2
	R(A)
	W(A)
R(A)	
W(A)	

UNIT V:

- Transaction Concept: Transaction State, ACID properties, Concurrent Executions, Serializability, **Recoverability**, Implementation of Isolation, Testing for Serializability, lock based, time stamp based, optimistic, concurrency protocols, Deadlocks, Failure Classification, Storage, Recovery and Atomicity, Recovery algorithm.
- Introduction to Indexing Techniques: B+ Trees, operations on B+Trees, Hash Based Indexing

Recoverability

- A recoverable schedule in DBMS ensures that the system can safely recover to a consistent state even if a transaction fails.
- This means a transaction can only commit after all transactions it depends on (i.e., has read from) have also committed.
- If a transaction T_i fails and rolls back, any transaction T_j that read T_i 's uncommitted data must also be rolled back to prevent data inconsistencies.

Recoverability

- **Irrecoverable Schedules:** An irrecoverable schedule occurs when a transaction commits after performing a dirty read—i.e., reading data written by another uncommitted transaction—and the original transaction later fails or is rolled back. This leads to inconsistency, as the committed transaction has used invalid data.

T1	T1's buffer space	T2	T2's Buffer Space	Database
				A=5000
R(A);	A=5000			A=5000
A=A-100;	A=4000			A=5000
W(A);	A=4000			A=4000
		R(A);	A=4000	A=4000
		A=A+500;	A=4500	A=4000
		W(A);	A=4500	A=4500
		Commit;		
Failure Point				
Commit;				

Recoverability

□ Recoverable with Cascading Rollback:

- A recoverable schedule with cascading rollback occurs when a failure in one transaction (T_i) causes other dependent transactions (T_j) to also roll back, but no committed transaction is affected.
- Since T_j reads data written by T_i and has not yet committed, it can be safely rolled back when T_i fails.

Recoverability

T1	T1's buffer space	T2	T2's Buffer Space	Database
				A=5000
R(A);	A=5000			A=5000
A=A-100;	A=4000			A=5000
W(A);	A=4000			A=4000
		R(A);	A=4000	A=4000
		A=A+500;	A=4500	A=4000
		W(A);	A=4500	A=4500
Failure Point				
Commit;				
		Commit;		

Recoverability

- **Cascadeless Recoverable Rollback:** A cascadeless recoverable schedule is a type of transaction schedule where **transactions are both recoverable and free from cascading rollbacks**. In this schedule, a transaction (**T_i**) reads data written by another transaction (**T_i**) only after **T_i** has committed.

T1	T1's buffer space	T2	T2's Buffer Space	Database
				A=5000
R(A);	A=5000			A=5000
A=A-100;	A=4000			A=5000
W(A);	A=4000			A=4000
Commit;				
		R(A);	A=4000	A=4000
		A=A+500;	A=4500	A=4000
		W(A);	A=4500	A=4500
		Commit;		

UNIT V:

- Transaction Concept: Transaction State, ACID properties, Concurrent Executions, Serializability, Recoverability, **Implementation of Isolation**, Testing for Serializability, lock based, time stamp based, optimistic, concurrency protocols, Deadlocks, Failure Classification, Storage, Recovery and Atomicity, Recovery algorithm.
- Introduction to Indexing Techniques: B+ Trees, operations on B+Trees, Hash Based Indexing

Implementation of Isolation

- DBMS implements transaction isolation **primarily through locking mechanisms**, which grant shared (read) or exclusive (write) access to data.
- **Timestamp-based protocols** are another method, assigning timestamps to transactions to enforce an execution order.
- The specific implementation of these techniques **varies by DBMS**, but they ultimately determine how transactions interact **to prevent concurrency problems like dirty reads, non-repeatable reads, and phantom reads**, thereby ensuring data.

UNIT V:

- Transaction Concept: Transaction State, ACID properties, Concurrent Executions, Serializability, Recoverability, Implementation of Isolation, Testing for Serializability, **lock based**, time stamp based, optimistic, concurrency protocols, Deadlocks, Failure Classification, Storage, Recovery and Atomicity, Recovery algorithm.
- Introduction to Indexing Techniques: B+ Trees, operations on B+Trees, Hash Based Indexing

Lock based concurrency protocols

- Introduction
- Types of Lock

Lock based concurrency protocols

- Introduction
- Types of Lock

Locking: Introduction

- A lock guarantees **exclusive use of a data item** to a current transaction.
- In other words, transaction **T2 does not have access to a data item that is currently being used by transaction T1.**
- A transaction **acquires a lock prior to data access.**
- The lock is **released (unlocked) when the transaction is completed** so that another transaction can lock the data item for its exclusive use.

Lock based concurrency protocols

- Introduction
- Types of Lock

Lock based concurrency protocols

- The DBMS may use different lock types:
- Binary Locks
- Shared/Exclusive Locks
- Locking Protocols

Lock based concurrency protocols

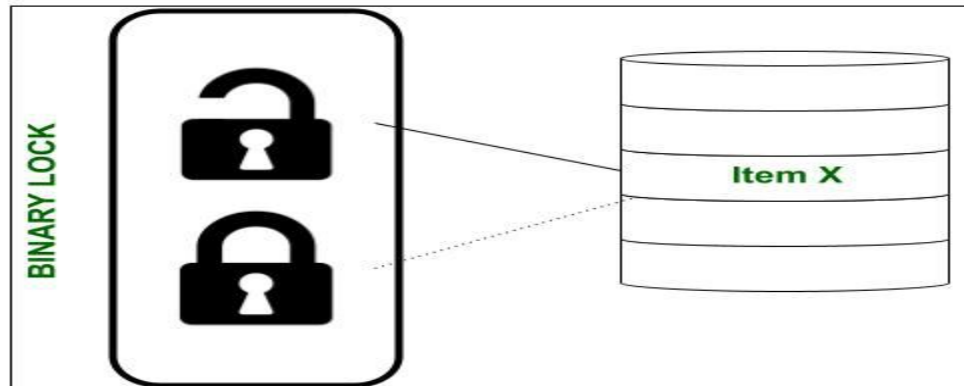
- The DBMS may use different lock types:
- **Binary Locks**
- Shared/Exclusive Locks
- Locking Protocols

Lock based concurrency protocols

- Binary Locks- Lock with 2 States
 - Locked -No other transaction can use that object
 - Unlocked -Any transaction can lock and use object
- All Transactions require a Lock and Unlock Operation for Each Object Accessed.
- ✓ Eliminates Lost Updates
- ✓ Too Restrictive to Yield Optimal Concurrency Conditions

Lock based concurrency protocols

- A transaction must issue the operation **lock()** before any read or write operations are performed in T.
- A transaction T must issue the operation **unlock** after all read and write operations are completed in T.



Lock based concurrency protocols

An Example of a Binary Lock

TABLE 9.10 AN EXAMPLE OF A BINARY LOCK

TIME	TRANSACTION	STEP	STORED VALUE
1	T1	Lock PRODUCT	
2	T1	Read PROD_QOH	15
3	T1	$\text{PROD_QOH} = 15 + 10$	
4	T1	Write PROD_QOH	25
5	T1	Unlock PRODUCT	
6	T2	Lock PRODUCT	
7	T2	Read PROD_QOH	23
8	T2	$\text{PROD_QOH} = 23 - 10$	
9	T2	Write PROD_QOH	13
10	T2	Unlock PRODUCT	

Lock based concurrency protocols

- The DBMS may use different lock types:
- Binary Locks
- Shared/Exclusive Locks
- Locking Protocols

Lock based concurrency protocols

- ❑ The binary lock is **too restrictive** for data items because at most one transaction can hold on a given item whether the transaction is reading or writing.
- ❑ To improve it we have shared and exclusive locks in which **more than one transaction can access the same item for reading purposes.**
- ❑ In this types of lock, system supports two kinds of lock :
 - ❑ **Exclusive(or Write) Locks**
 - ❑ **Shared(or Read) Locks.**

Lock based concurrency protocols

- ❑ Exclusive locks are placed on resources whenever a write operation (INSERT, UPDATE And DELETE) are performed.
- ❑ Multiple shared locks can be placed on a resource.
- ❑ **Only one exclusive lock** can be placed on a resource at a time.
- ❑ The first **user who acquires an exclusive lock will continue to have the sole ownership** of the resource, and no other user can acquire an exclusive lock on that resource.

Lock based concurrency protocols

- If a transaction T_i has locked the data item A in **shared mode**, then a request from another transaction T_j on A for :
 - Write operation on A : **Denied**. Transaction T_j has to wait until transaction T_i unlock A.
 - Read operation on A : **Allowed**.
- If a transaction T_i has locked a data item a in **exclusive mode** then request from some another transaction T_j for
 - Read operation on A : **Denied**
 - Write operation on A : **Denied**

Lock based concurrency protocols

- The relationship between Shared and Exclusive Lock can be represented by the following table Which is known as Lock Matrix

		Locks already existing	
Locks to Be granted		Shared	Exclusive
	Shared	TRUE	FALSE
	Exclusive	FALSE	FALSE

Lock based concurrency protocols

- The DBMS may use different lock types:
- Binary Locks
- Shared/Exclusive Locks
- Locking Protocols (Multi version concurrency control)

Lock based concurrency protocols

- Two Phase Locking
- Strict Two Phase Locking
- Rigorous two phase locking protocol

Lock based concurrency protocols

- Two Phase Locking
- Strict Two Phase Locking
- Rigorous two phase locking protocol

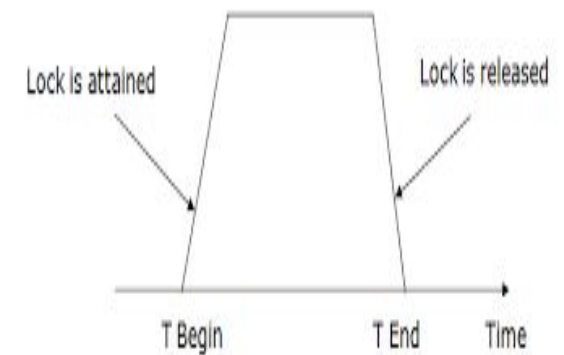
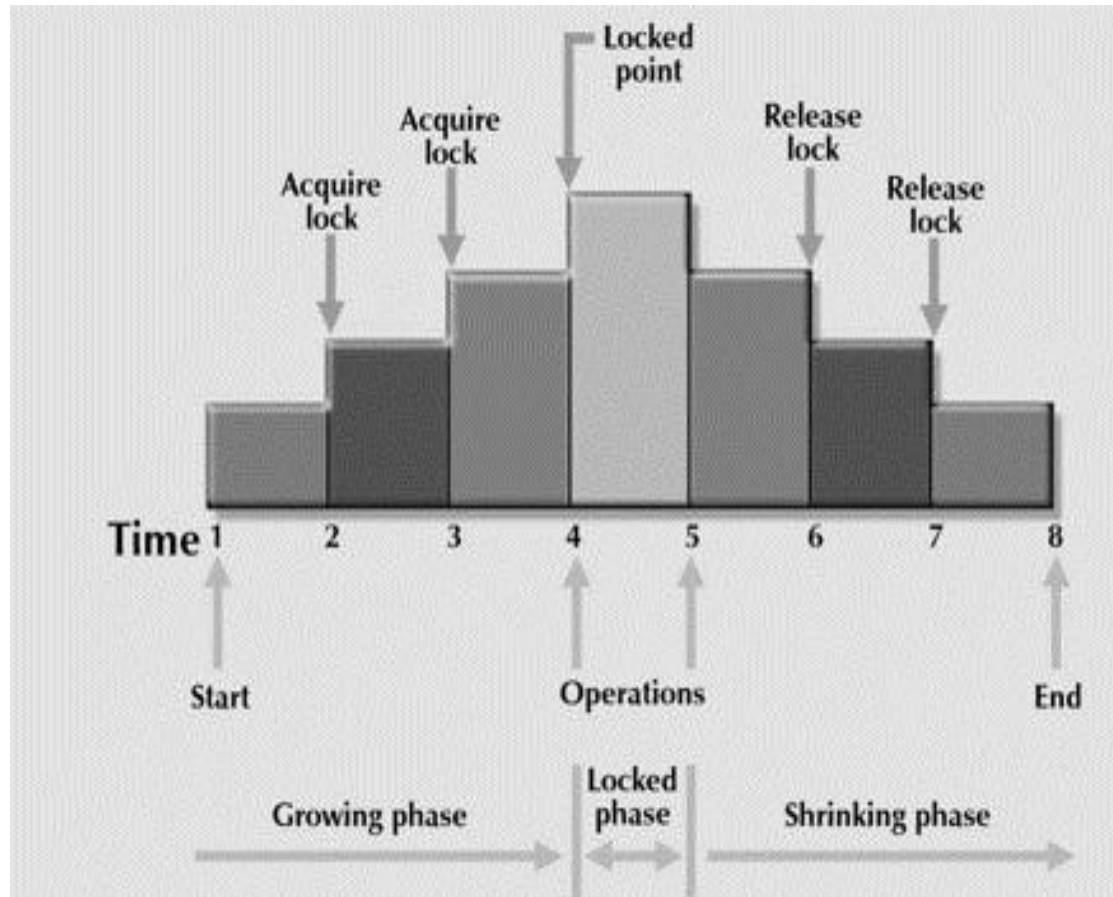
Lock based concurrency protocols

- The use of locks has helped us to **create neat and clean concurrent schedule.**
- The Two Phase Locking Protocol defines the **rules of how to acquire the locks** on a data item and how to release the locks.
- The Two Phase Locking Protocol assumes that a transaction can only be in one of two phases.
 - **Growing Phase**
 - **Shrinking Phase**

Lock based concurrency protocols

- **Growing Phase** – All the locks are issued in this phase. No locks are released, after all changes to data-items are committed and then the second phase (shrinking phase) starts.
- **Shrinking phase** – No locks are issued in this phase, all the changes to data-items are noted (stored) and then locks are released.
- The 2PL locking protocol is represented diagrammatically as follows –

Lock based concurrency protocols



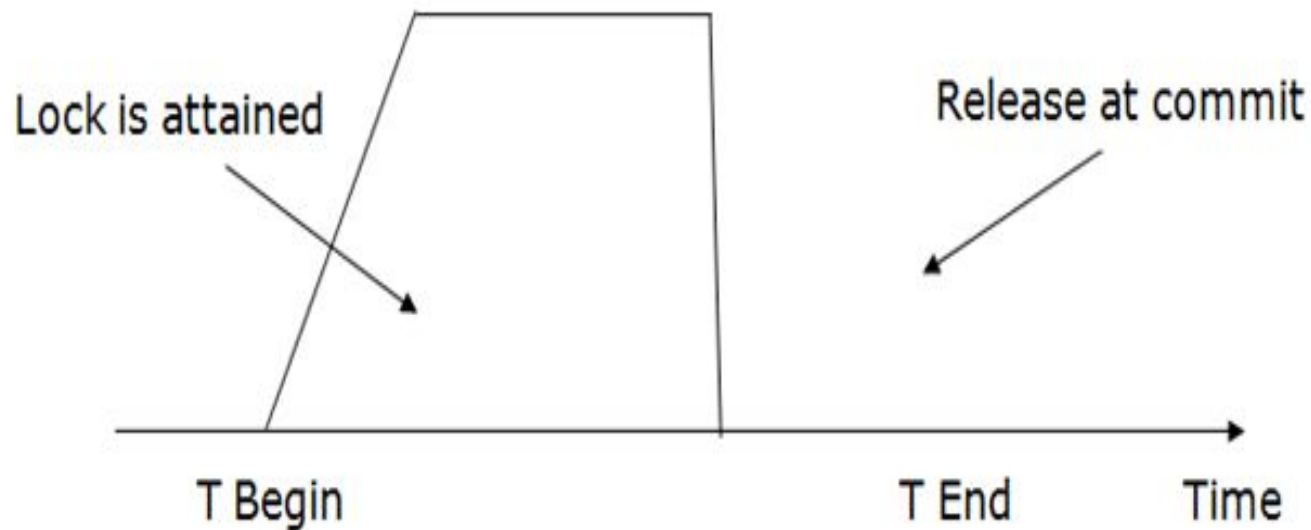
Lock based concurrency protocols

- Two Phase Locking
- Strict Two Phase Locking
- Rigorous two phase locking protocol

Lock based concurrency protocols

- The first phase of Strict-2PL is same as 2PL.
- After acquiring all the locks in the first phase, the transaction continues to execute normally.
- But in contrast to 2PL, A transaction **can release a shared lock after the lock point**, but it **cannot release any exclusive lock** until the transaction commits.
- Strict-2PL holds all the exclusive locks until the commit point and releases all the exclusive locks at a time.

Lock based concurrency protocols



Lock based concurrency protocols

- Two Phase Locking
- Strict Two Phase Locking
- Rigorous two phase locking protocol

Lock based concurrency protocols

- This protocol requires that all the **shared and exclusive locks** to be held until the transaction commits.
- In Rigorous Two Phase Locking Protocol, **a transaction is not allowed to release any lock (either shared or exclusive) until it commits.**

UNIT V:

- Transaction Concept: Transaction State, ACID properties, Concurrent Executions, Serializability, Recoverability, Implementation of Isolation, Testing for Serializability, lock based, **time stamp based, optimistic, concurrency protocols**, Deadlocks, Failure Classification, Storage, Recovery and Atomicity, Recovery algorithm.
- Introduction to Indexing Techniques: B+ Trees, operations on B+Trees, Hash Based Indexing

Timestamp based optimistic concurrency protocols

- Timestamp based Protocol in DBMS is an algorithm which uses the **System Time or Logical Counter** as a timestamp to serialize the execution of concurrent transactions.
- The Timestamp-based protocol ensures that every **conflicting read and write operations are executed in a timestamp order.**
- The **older transaction is always given priority** in this method.
- It uses **system time to determine the time stamp** of the transaction.
- This is the **most commonly used concurrency protocol.**

Timestamp based optimistic concurrency protocols

- Suppose there are there transactions T1, T2, and T3.
- T1 has entered the system at time 0010
- T2 has entered the system at 0020
- T3 has entered the system at 0030
- Priority will be given to transaction T1, then transaction T2 and lastly Transaction T3.

Timestamp based optimistic concurrency protocols

- $TS(T_i)$ denotes the timestamp of the transaction T_i .
- $RTS(X)$ denotes the Read time-stamp of data-item X .
- $WTS(X)$ denotes the Write time-stamp of data-item X .

Timestamp based optimistic concurrency protocols

- Timestamp ordering protocol works as follows:
- If a transaction T_i issues a read(X) operation:
 - If $WTS(X) > TS(T_i)$ Operation rejected.
 - Otherwise set $RTS(X) = \max(RTS(X), TS(T_i))$.
- If a transaction T_i issues a write(X) operation:
 - If $RTS(X) > TS(T_i)$ Operation rejected.
 - If $WTS(X) > TS(T_i)$ Operation rejected and T_i rolled back.
 - Otherwise set $WTS(X) = TS(T_i)$.

Timestamp based optimistic concurrency protocols

T1(100)	T2(200)	T3(300)
R(A)		
	R(B)	
W(C)		
		R(B)
R(C)		
	W(B)	
		W(A)

	A	B	C
RTS	0	0	0
WTS	0	0	0

Timestamp based optimistic concurrency protocols

T1(100)	T2(200)	T3(300)
R(A)		
	R(B)	
W(C)		
		R(B)
R(C)		
	W(B)	
		W(A)

	A	B	C
RTS	0	0	0
WTS	0	0	0

Timestamp based optimistic concurrency protocols

- $R(A)$
- If a transaction T_i issues a $\text{read}(X)$ operation:
 - If $WTS(X) > TS(T_i)$ Operation rejected.
 - Otherwise set $RTS(X) = \max(RTS(X), TS(T_i))$.

Timestamp based optimistic concurrency protocols

- $R(A)$
- If a transaction $T1$ issues a read(A) operation:
 - If $WTS(A) > TS(T1)$ Operation rejected.
 - Otherwise set $RTS(A) = \max(RTS(A), TS(T1))$.
- $WTS(A)=0, TS(T1)=100$

	A	B	C
RTS	0	0	0
WTS	0	0	0

Timestamp based optimistic concurrency protocols

- $R(A)$
- If a transaction $T1$ issues a read(A) operation:
 - If $WTS(A) > TS(T1)$ Operation rejected.
 - Otherwise set $RTS(A) = \max(RTS(A), TS(T1))$.
- $WTS(A)=0, TS(T1)=100$

	A	B	C
RTS	100	0	0
WTS	0	0	0

Timestamp based optimistic concurrency protocols

T1(100)	T2(200)	T3(300)
R(A)		
	R(B)	
W(C)		
		R(B)
R(C)		
	W(B)	
		W(A)

	A	B	C
RTS	100	0	0
WTS	0	0	0

Timestamp based optimistic concurrency protocols

- $R(B)$
 - If a transaction $T2$ issues a $read(B)$ operation:
 - If $WTS(B) > TS(T2)$ Operation rejected.
 - Otherwise set $RTS(B) = \max(RTS(B), TS(T2))$.
- $WTS(B)=0, TS(T2)=200$

	A	B	C
RTS	100	0	0
WTS	0	0	0

Timestamp based optimistic concurrency protocols

- $R(B)$
- If a transaction $T2$ issues a $read(B)$ operation:
- If $WTS(B) > TS(T2)$ Operation rejected.
- Otherwise set $RTS(B) = \max(RTS(B), TS(T2))$.
 $WTS(B)=0, TS(T2)=200$

	A	B	C
RTS	100	200	0
WTS	0	0	0

Timestamp based optimistic concurrency protocols

T1(100)	T2(200)	T3(300)
R(A)		
	R(B)	
W(C)		
		R(B)
R(C)		
	W(B)	
		W(A)

	A	B	C
RTS	100	200	0
WTS	0	0	0

Timestamp based optimistic concurrency protocols

- $W(C)$:
- If a transaction $T1$ issues a $write(C)$ operation:
- If $RTS(C) > TS(T1)$ Operation rejected.
- If $WTS(C) > TS(T1)$ Operation rejected and Ti rolled back.
- Otherwise set $WTS(C) = TS(T1)$.
- $RTS(C)=0, WTS(C)=0, TS(T1)=100$.

	A	B	C
RTS	100	200	0
WTS	0	0	0

Timestamp based optimistic concurrency protocols

- $W(C)$:
- If a transaction $T1$ issues a $write(C)$ operation:
- If $RTS(C) > TS(T1)$ Operation rejected.
- If $WTS(C) > TS(T1)$ Operation rejected and Ti rolled back.
- Otherwise set $WTS(C) = TS(T1)$.
- $RTS(C)=0, WTS(C)=0, TS(T1)=100$.

	A	B	C
RTS	100	200	0
WTS	0	0	100

Timestamp based optimistic concurrency protocols

T1(100)	T2(200)	T3(300)
R(A)		
	R(B)	
W(C)		
		R(B)
R(C)		
	W(B)	
		W(A)

	A	B	C
RTS	100	200	0
WTS	0	0	100

Timestamp based optimistic concurrency protocols

- $R(B)$:
 - If a transaction $T3$ issues a $\text{read}(B)$ operation:
 - If $WTS(B) > TS(T3)$ Operation rejected.
 - Otherwise set $RTS(B) = \max(RTS(B), TS(T3))$.
- $WTS(B)=0, TS(T3)=300, RTS(B)=200$

	A	B	C
RTS	100	200	0
WTS	0	0	100

Timestamp based optimistic concurrency protocols

- $R(B)$:
 - If a transaction $T3$ issues a $\text{read}(B)$ operation:
 - If $WTS(B) > TS(T3)$ Operation rejected.
 - Otherwise set $RTS(B) = \max(RTS(B), TS(T3))$.
- $WTS(B)=0, TS(T3)=300, RTS(B)=200$

	A	B	C
RTS	100	300	0
WTS	0	0	100

Timestamp based optimistic concurrency protocols

T1(100)	T2(200)	T3(300)
R(A)		
	R(B)	
W(C)		
		R(B)
R(C)		
	W(B)	
		W(A)

	A	B	C
RTS	100	300	0
WTS	0	0	100

Timestamp based optimistic concurrency protocols

- $R(C)$
- If a transaction $T1$ issues a $read(C)$ operation:
- If $WTS(C) > TS(T1)$ Operation rejected.
- Otherwise set $RTS(C) = \max (RTS(C), TS (T1))$.
 $WTS(C)=100, TS(T1)=100, RTS(C)=0$

	A	B	C
RTS	100	300	0
WTS	0	0	100

Timestamp based optimistic concurrency protocols

- $R(C)$
- If a transaction $T1$ issues a $read(C)$ operation:
- If $WTS(C) > TS(T1)$ Operation rejected.
- Otherwise set $RTS(C) = \max(RTS(C), TS(T1))$.
 $WTS(C)=100, TS(T1)=100, RTS(C)=0$

	A	B	C
RTS	100	300	100
WTS	0	0	100

Timestamp based optimistic concurrency protocols

T1(100)	T2(200)	T3(300)
R(A)		
	R(B)	
W(C)		
		R(B)
R(C)		
	W(B)	
		W(A)

	A	B	C
RTS	100	300	100
WTS	0	0	100

Timestamp based optimistic concurrency protocols

- $W(B)$
- If a transaction $T2$ issues a write(B) operation:
 - If $RTS(B) > TS(T2)$ Operation rejected.
 - If $WTS(B) > TS(T2)$ Operation rejected and T_i rolled back.
 - Otherwise set $WTS(B) = TS(T2)$.
- $RTS(B)=300, WTS(B)=0, TS(T2)=200$

Timestamp based optimistic concurrency protocols

- $W(B)$
- If a transaction $T2$ issues a write(B) operation:
 - If $RTS(B) > TS(T2)$ Operation rejected.
 - If $WTS(B) > TS(T2)$ Operation rejected and T_i rolled back.
 - Otherwise set $WTS(B) = TS(T2)$.
- $RTS(B)=300, WTS(B)=0, TS(T2)=200$

Timestamp based optimistic concurrency protocols

T1(100)	T2(200)	T3(300)
R(A)		
	R(B)	
W(C)		
		R(B)
R(C)		
	W(B)=Rejected	
		W(A)

	A	B	C
RTS	100	300	100
WTS	0	0	100

Timestamp based optimistic concurrency protocols

T1(100)	T2(200)	T3(300)
R(A)		
	R(B)	
W(C)		
		R(B)
R(C)		
	W(B)=Rejected	
		W(A)

	A	B	C
RTS	100	300	100
WTS	0	0	100

Timestamp based optimistic concurrency protocols

- $W(A)$
- If a transaction $T3$ issues a $\text{write}(A)$ operation:
 - If $\text{RTS}(A) > \text{TS}(T3)$ Operation rejected.
 - If $\text{WTS}(A) > \text{TS}(T3)$ Operation rejected and T_i rolled back.
 - Otherwise set $\text{WTS}(A) = \text{TS}(T3)$.
- $\text{RTS}(A)=100, \text{WTS}(A)=0, \text{TS}(T3)=300$

	A	B	C
RTS	100	300	100
WTS	0	0	100

Timestamp based optimistic concurrency protocols

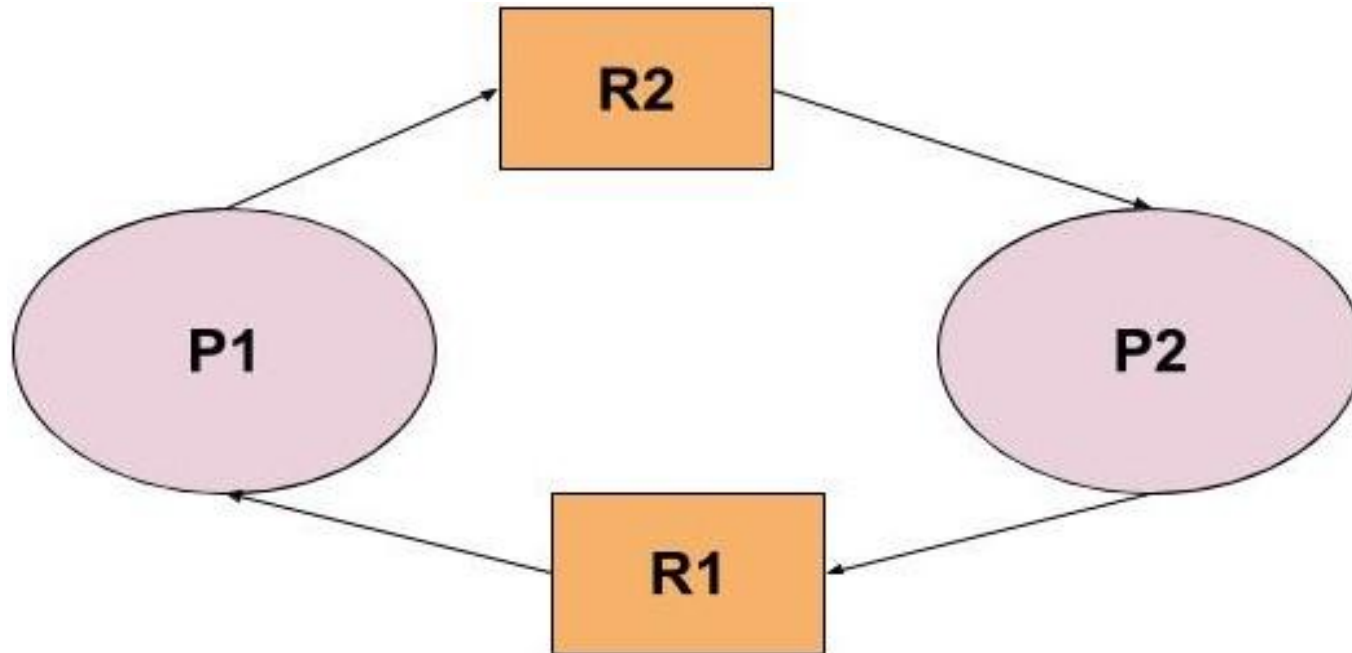
- $W(A)$
- If a transaction $T3$ issues a $\text{write}(A)$ operation:
 - If $\text{RTS}(A) > \text{TS}(T3)$ Operation rejected.
 - If $\text{WTS}(A) > \text{TS}(T3)$ Operation rejected and T_i rolled back.
 - Otherwise set $\text{WTS}(A) = \text{TS}(T3)$.
- $\text{RTS}(A)=100, \text{WTS}(A)=0, \text{TS}(T3)=300$

	A	B	C
RTS	100	300	100
WTS	300	0	100

UNIT V:

- Transaction Concept: Transaction State, ACID properties, Concurrent Executions, Serializability, Recoverability, Implementation of Isolation, Testing for Serializability, lock based, time stamp based, optimistic, concurrency protocols, **Deadlocks**, Failure Classification, Storage, Recovery and Atomicity, Recovery algorithm.
- Introduction to Indexing Techniques: B+ Trees, operations on B+Trees, Hash Based Indexing

Deadlocks- Introduction



**Process P1 is holding R1 and waiting for R2 while
Process P2 is holding R2 and waiting for R1.**

Deadlocks- Introduction

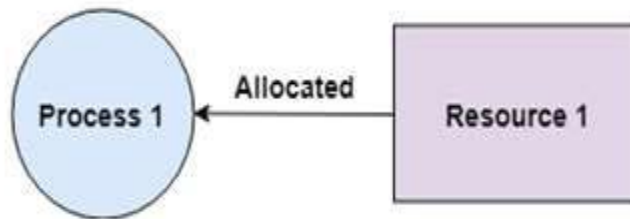
- A deadlock occurs in a multi-user database environment when two or more transactions block each other indefinitely by each holding a resource the other needs.
- In Deadlock, one process is holding some resource and is waiting for some other resource and at the same time, there is some other process that is holding that resource and is waiting for the resources held by the first process.
- So, both the processes are waiting for each other to release the resources and none of them are releasing the resources.

Deadlocks- Necessary Conditions

- Coffman stated four conditions for a deadlock occurrence. A deadlock may occur if all the following conditions holds true.
- 1. Mutual exclusion condition
- 2. Hold and wait condition
- 3. No preemption condition
- 4. Circular wait condition

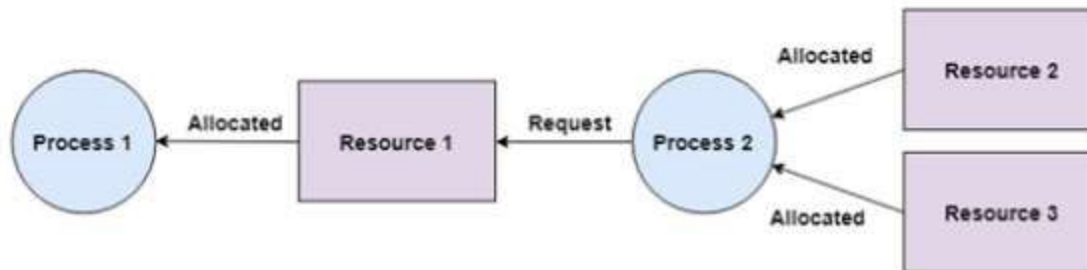
Deadlocks- Necessary Conditions

- **Mutual exclusion:**
- A resource can only be shared in mutually exclusive manner. It implies that, **two process cannot use the same resource at the same time.**
- In the diagram below, there is a single instance of Resource 1 and it is held by Process 1 only.



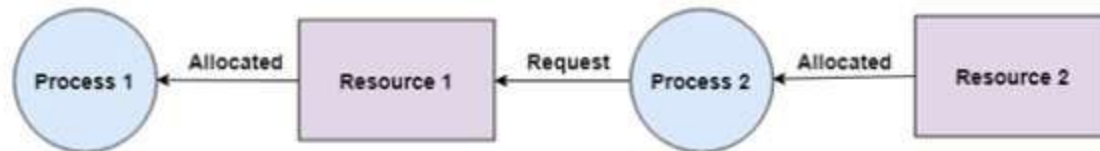
Deadlocks- Necessary Conditions

- **Hold and wait**
- A process **can hold multiple resources and still request more resources** from other processes which are holding them.
- In the diagram given below, Process 2 holds Resource 2 and Resource 3 and is requesting the Resource 1 which is held by Process 1.



Deadlocks- Necessary Conditions

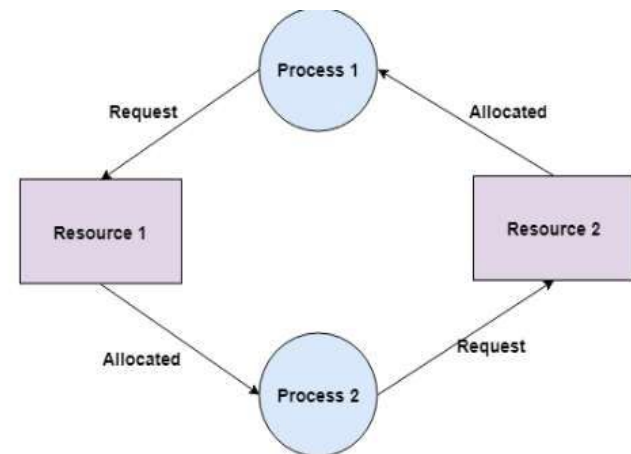
- **No preemption**
- A resource cannot be preempted from a process by force. **A process can only release a resource voluntarily.**
- In the diagram below, Process 2 cannot preempt Resource 1 from Process 1. It will only be released when Process 1 relinquishes it voluntarily after its execution is complete.



Deadlocks- Necessary Conditions

□ Circular wait

- All the **processes must be waiting for the resources in a cyclic manner** so that the last process is waiting for the resource which is being held by the first process.
- For example: Process 1 is allocated Resource 2 and it is requesting Resource 1. Similarly, Process 2 is allocated Resource 1 and it is requesting Resource 2. This forms a circular wait loop.



Deadlocks-Detection

- The DBMS periodically tests the database for deadlocks.
- If a deadlock is found, **one of the transactions** (the “victim”) **is aborted** (rolled back and restarted) **and the other transaction continues**.
- **There are two algorithms for deadlock avoidance.**
 - Wait/Die
 - Wound/Wait
- **Timestamp assignment:** Each transaction is assigned a unique timestamp when it begins. The older the transaction, the lower its timestamp.

Deadlocks-Detection-**Wait-Die**

- **Requesting a locked resource:** When a transaction (T1) requests a resource that is already locked by another transaction (T2), the system compares their timestamps.
- **Older transaction (T1) waits:** If T1's timestamp is smaller than T2's (T1 is older), T1 waits for T2 to release the resource.
- **Younger transaction (T1) dies:** If T1's timestamp is larger than T2's (T1 is younger), T1 is aborted and killed. It will be restarted later with the same timestamp, giving it a chance to acquire the resource when the other transaction has finished.

Deadlocks-Detection-Wound-Wait

- **Request Lock:** When a transaction (T1) requests a resource held by another transaction (T2), the timestamps are compared.
- **Older Transaction Request (Wound):** If T1 is older (smaller timestamp) and requests a resource held by the younger T2, T1 "wounds" T2.
 - T2 is aborted and restarted.
 - T1 then acquires the resource.
- **Younger Transaction Request (Wait):**
- If T1 is younger (larger timestamp) and requests a resource held by the older T2, T1 waits for T2 to release the resource.

UNIT V:

- Transaction Concept: Transaction State, ACID properties, Concurrent Executions, Serializability, Recoverability, Implementation of Isolation, Testing for Serializability, lock based, time stamp based, optimistic, concurrency protocols, Deadlocks, **Failure Classification**, Storage, Recovery and Atomicity, Recovery algorithm.
- Introduction to Indexing Techniques: B+ Trees, operations on B+Trees, Hash Based Indexing

Failure Classification

- DBMS failure classification categorizes unexpected events into types like:
- **Transaction Failure:** which includes logical and system errors within a transaction.
- **System Failure:** a crash of the DBMS or OS losing volatile memory.
- **Disk Failure:** where permanent storage (like disks) is corrupted or lost.

Failure Classification

- **1. Transaction Failure**
- **Logical Errors:** Occur due to errors within the transaction itself, such as invalid input, constraint violations (e.g., data not found, exceeding resource limits), or division by zero.
- **System Errors:** The DBMS itself terminates a transaction due to an undesirable state, like a deadlock or a lack of system resources.

Failure Classification

- **2. System Failure (Soft Failure)**
- Occurs due to hardware malfunctions (like power outages) or software bugs that cause the database system to crash.
- The contents of volatile storage (like RAM) are lost, but non-volatile storage (disks) is generally intact.
- Recovery typically involves using transaction logs and checkpoints to restore consistency.

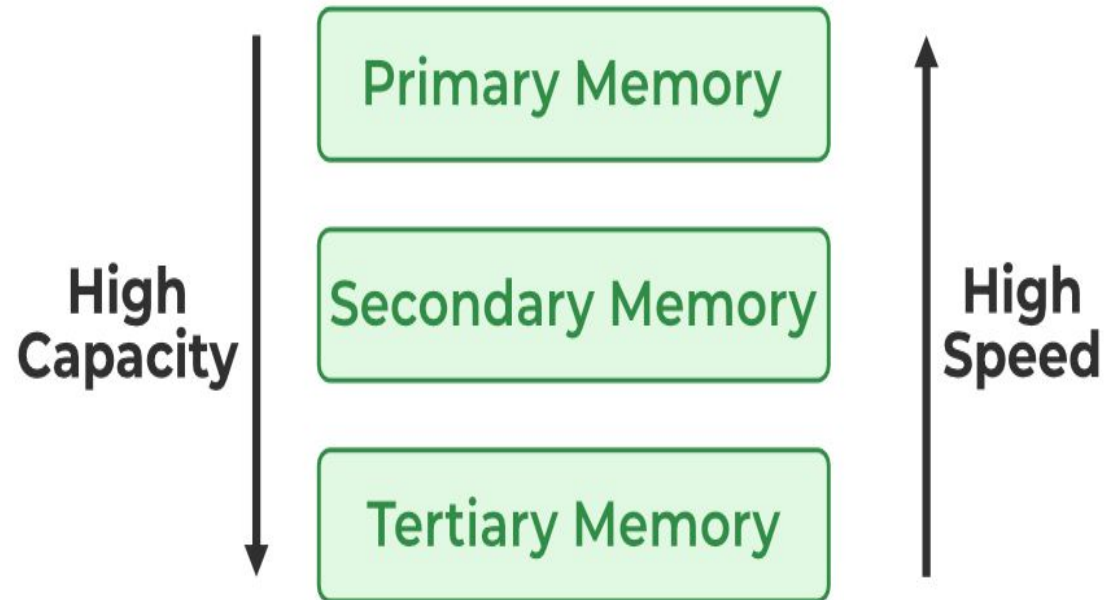
Failure Classification

- **3. Disk Failure (Hard Failure)**
- Involves physical damage or corruption to the non-volatile storage media (like hard drives or SSDs).
- This can lead to data corruption or complete loss of data from that storage medium.
- Requires proper backup and recovery mechanisms to restore the database to a previous stable state.

UNIT V:

- Transaction Concept: Transaction State, ACID properties, Concurrent Executions, Serializability, Recoverability, Implementation of Isolation, Testing for Serializability, lock based, time stamp based, optimistic, concurrency protocols, Deadlocks, Failure Classification, **Storage**, Recovery and Atomicity, Recovery algorithm.
- Introduction to Indexing Techniques: B+ Trees, operations on B+Trees, Hash Based Indexing

Storage



Storage

- In a Database Management System (DBMS), storage refers to **how data is physically organized, managed, and retrieved** from various storage media.
- The storage system is a core component **responsible for an interface between the data on the disk and the queries that access it.**
- The **main goal is to store data reliably and access it efficiently.**

Storage- Hierarchy

- **1.Primary storage:** This is the **fastest and most expensive storage**, located **closest to the CPU**. It is **volatile**, meaning data is lost when the power is turned off.
- **Registers:** The fastest and smallest memory, located inside the CPU. They hold data that is actively being processed.
- **Cache:** A small, fast memory that stores frequently accessed data to reduce the need to access the main memory.
- **Main memory (RAM):** Holds the data that the DBMS is currently operating on. The buffer manager is responsible for fetching data pages from disk into the main memory.

Storage- Hierarchy

- **2. Secondary storage:** This is the **main, non-volatile** storage for the database. It is **slower and cheaper than primary storage** but can hold a much larger amount of data permanently.
 - **Magnetic disks (Hard Disk Drives - HDDs):** The most common secondary storage device, offering a balance of performance and cost. Data is organized into tracks, sectors, and cylinders.
 - **Solid-State Drives (SSDs):** Faster than HDDs with no moving parts, offering significantly better performance for frequent read/write operations.
 - **RAID (Redundant Array of Independent Disks):** A technology that uses multiple disk drives to increase performance, provide data redundancy, or both.

Storage- Hierarchy

- **3. Tertiary storage:** This is the **slowest, cheapest, and highest-capacity storage**, used mainly for backups and long-term archival. Data access is often sequential.
 - **Magnetic tapes:** Used for creating backups of massive datasets.
 - **Optical disks:** Such as DVDs and CDs, can store large amounts of data cheaply.

UNIT V:

- Transaction Concept: Transaction State, ACID properties, Concurrent Executions, Serializability, Recoverability, Implementation of Isolation, Testing for Serializability, lock based, time stamp based, optimistic, concurrency protocols, Deadlocks, Failure Classification, Storage, **Recovery and Atomicity, Recovery algorithm.**
- Introduction to Indexing Techniques: B+ Trees, operations on B+Trees, Hash Based Indexing

Recovery and Atomicity, Recovery algorithm

- Introduction
- Log Based Recovery

Recovery and Atomicity, Recovery algorithm

- Introduction
- Log Based Recovery

Recovery and Atomicity, Recovery algorithm

- DBMS is a highly complex system with hundreds of transactions being executed every second.
- If it fails or crashes amid transactions, it is **expected that the system would follow some sort of algorithm or techniques to recover lost data.**
- There are problems that may cause the system to stop abruptly and cause the system to crash.
- For example, interruptions in power supply may cause the failure of underlying hardware or software failure.

Recovery and Atomicity, Recovery algorithm

- When a system crashes, it may have **several transactions being executed and various files opened for them to modify the data items.**
- When a DBMS recovers from a crash, it should maintain the following
 - It should **check the states of all the transactions**, which were being executed.
 - A transaction may be in the **middle of some operation**; the DBMS must ensure the atomicity of the transaction in this case.
 - It should check whether the transaction can be completed now or it needs to be rolled back.

Recovery and Atomicity, Recovery algorithm

- There are **two types of techniques**, which can help a DBMS in recovering as well as maintaining the atomicity of a transaction
 - Maintaining the **logs of each transaction**, and writing them onto some stable storage before actually modifying the database.
 - Maintaining **shadow paging**, where the changes are done on a volatile memory, and later, the actual database is updated.

Recovery and Atomicity, Recovery algorithm

- Introduction
- Log Based Recovery

Recovery and Atomicity, Recovery algorithm

- Log is nothing but a file which contains a sequence of records, each log record refers to a write operation.
- All the log records are recorded step by step in the log file. We can say, log files store the history of all update activities.
- Log contains start of transaction, transaction number, record number, old value, new value, end of transaction etc. For example, mini statements in bank ATMs.

Recovery and Atomicity, Recovery algorithm



Recovery and Atomicity, Recovery algorithm

- If within an ongoing transaction, the system crashes, then by using log files, **we can return back to the previous state as if nothing has happened to the database.**
- The **log is kept on disk** so that it is not affected by failures except disk and failures.

Recovery and Atomicity, Recovery algorithm

- Different types of log records are as follows:
- $\langle T_i, X_i, V_1, V_2 \rangle$ update log record, where T_i =transaction, X_i =data, V_1 =old data, V_2 =new value.
- Transaction T_i starts **execution**.
- Transaction T_i is **committed**.
- Transaction T_i is **aborted**.

Recovery and Atomicity, Recovery algorithm

Create a log for the given transaction T1 and T2.

T1	T2	Log
Read A	Read A	<T1, start>
A=A-2000	A=A+5000	<T1,A,5000, 3000>
Write A	Write A	<T1, B, 8000, 10000>
Read B	Read B	<T1, commit>
B=B+2000	B= B+7000	<T2, start>
Write B	Write B	<T2, A, 3000, 8000>
		<T2, B, 10000, 17000>
		<T2, commit>

Recovery and Atomicity, Recovery algorithm

- After a system crash has occurred, the system consults the log to determine which transactions need to be redone and which need to be undone.
- Transaction T_i needs to be undone if the log contains the record $\langle T_i \text{ start} \rangle$ but does not contain either the record $\langle T_i \text{ commit} \rangle$ or the record $\langle T_i \text{ abort} \rangle$.
- Transaction T_i needs to be redone if log contains record $\langle T_i \text{ start} \rangle$ and the record $\langle T_i \text{ commit} \rangle$.

UNIT V:

- Transaction Concept: Transaction State, ACID properties, Concurrent Executions, Serializability, Recoverability, Implementation of Isolation, Testing for Serializability, lock based, time stamp based, optimistic, concurrency protocols, Deadlocks, Failure Classification, Storage, Recovery and Atomicity, Recovery algorithm.
- Introduction to Indexing Techniques: B+ Trees, operations on B+Trees, Hash Based Indexing

Introduction to Indexing Techniques

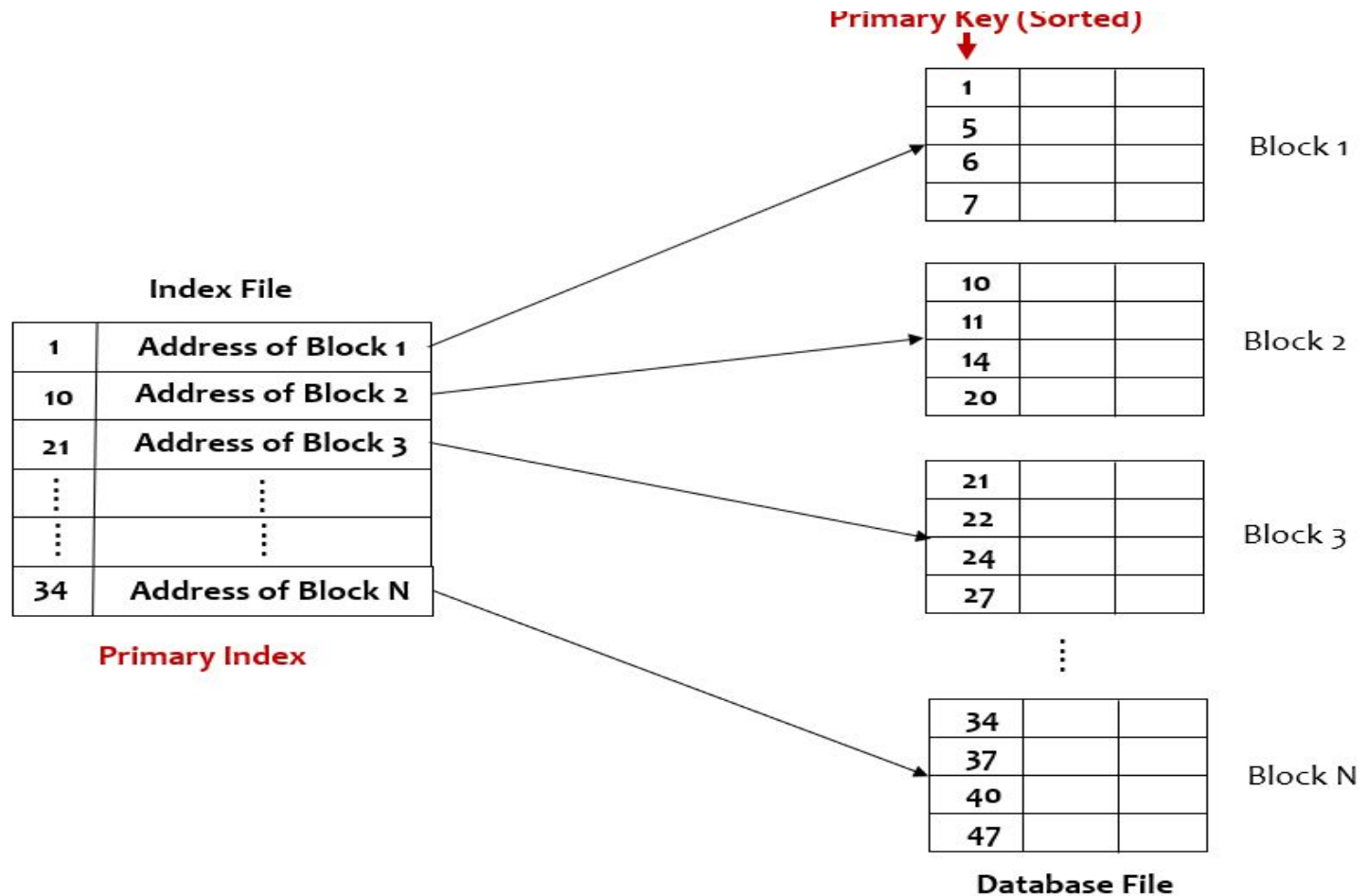
- Indexing is a **data structure technique to efficiently retrieve records from the database** files based on some attributes on which the indexing has been done.
- An index is a small table having only two columns.

Search Key	Data Reference
------------	----------------

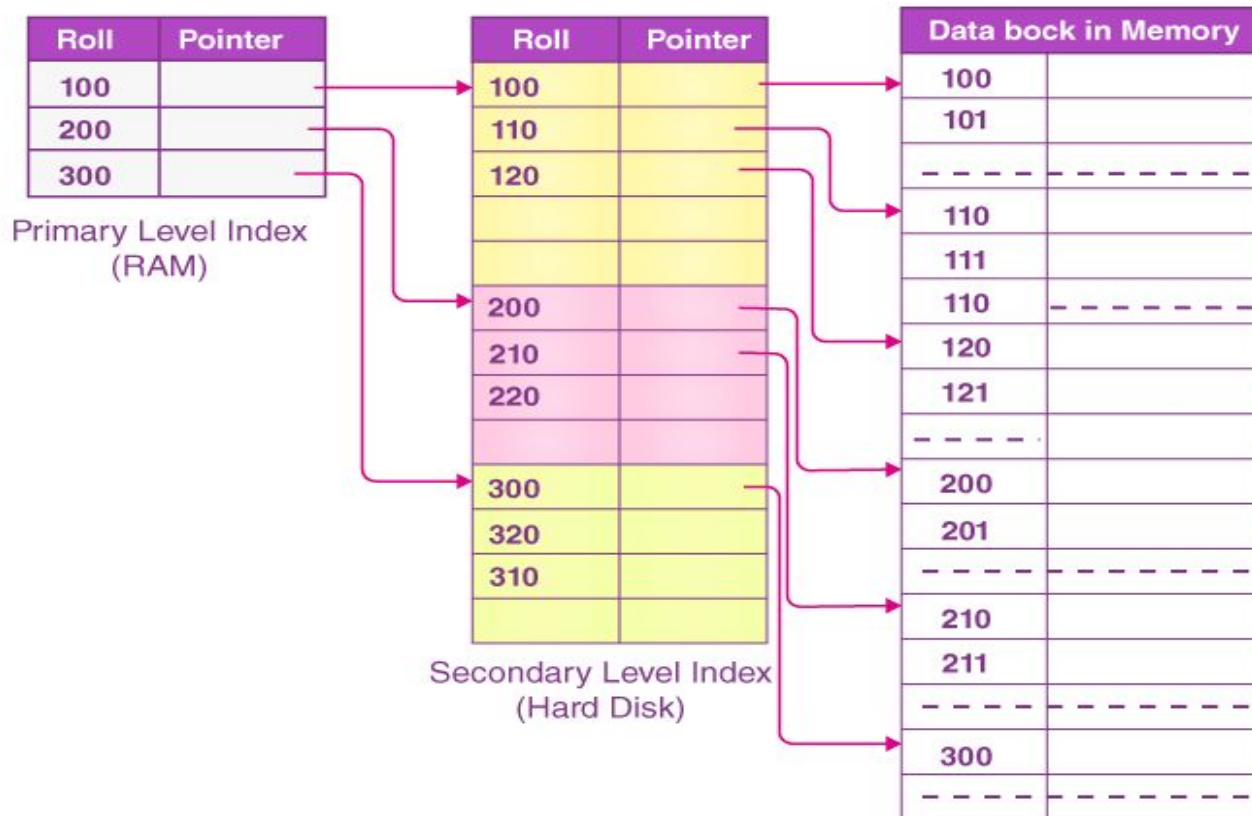
Structure of an index

- First column contains a **copy of the primary or candidate key** of a table.
- Second column contains a **set of pointers holding the address** of the disk block where that particular key value can be found.

Introduction to Indexing Techniques



Introduction to Indexing Techniques



UNIT V:

- Transaction Concept: Transaction State, ACID properties, Concurrent Executions, Serializability, Recoverability, Implementation of Isolation, Testing for Serializability, lock based, time stamp based, optimistic, concurrency protocols, Deadlocks, Failure Classification, Storage, Recovery and Atomicity, Recovery algorithm.
- Introduction to Indexing Techniques: B+ Trees, operations on B+Trees, Hash Based Indexing

Hash Based Indexing

- Introduction
- Types of Hashing
 - Static Hashing
 - Dynamic Hashing

Hash Based Indexing

- Introduction
- Types of Hashing
 - Static Hashing
 - Dynamic Hashing

Hash Based Indexing

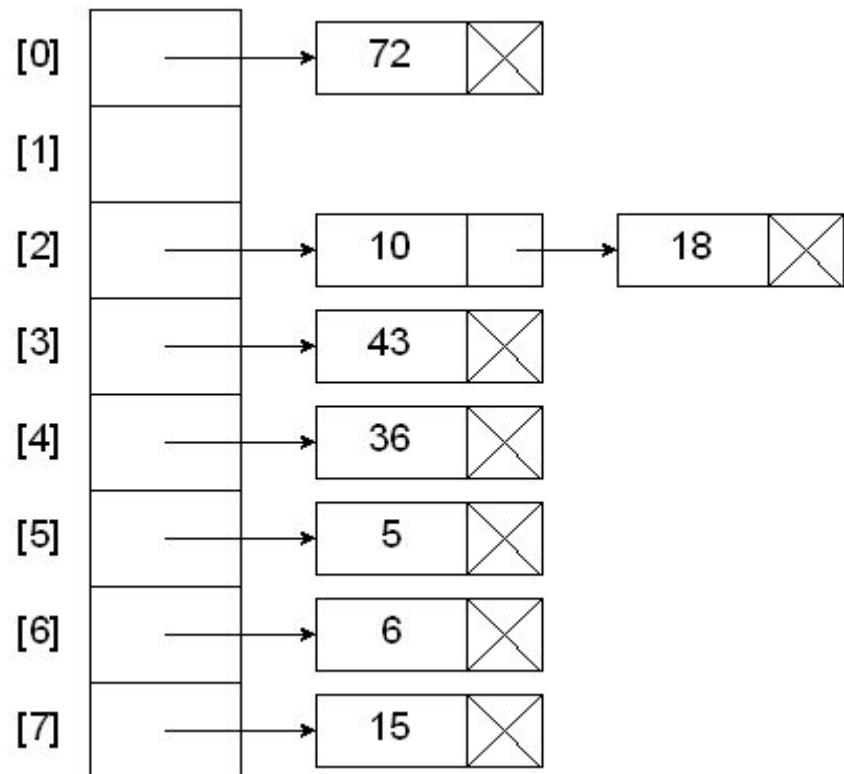
- Hash File organization method is the one where data is stored at the data blocks whose address is generated by using hash function.
- The memory location where these records are stored is called as data block or data bucket.
- This data bucket is capable of storing one or more records.
- The hash function can use any of the column value to generate the address.

$$h(k) = k \bmod m$$

Hash Based Indexing

Hash key = key % table size

4	=	36	%	8
2	=	18	%	8
0	=	72	%	8
3	=	43	%	8
6	=	6	%	8
2	=	10	%	8
5	=	5	%	8
7	=	15	%	8



Hash Based Indexing

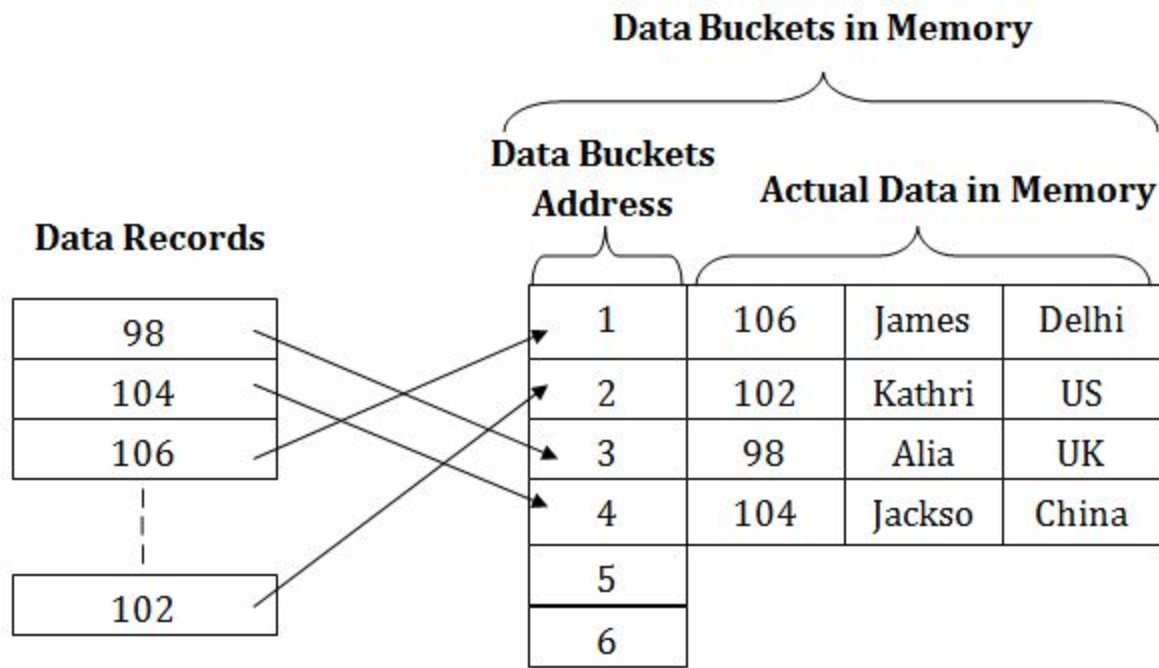
- Introduction
- Types of Hashing
 - Static Hashing
 - Dynamic Hashing

Hashing: Static Hashing

- In static hashing, the resultant data bucket address will always be the same.
- That means if we generate an address for EMP_ID = 103 using the hash function $\text{mod } 5$ then it will always result in same bucket address 3.
- Here, there will be no change in the bucket address.
- Hence in this static hashing, the number of data buckets in memory remains constant throughout.

Hashing: Static Hashing

- In this example, we will have data buckets in the memory used to store the data.



Hashing: Static Hashing

- If we want to insert some new records into the file but the data bucket address generated by the hash function is not empty or the data already exists in that address.
- This becomes a critical situation to handle.
- This situation in the static hashing is called **bucket overflow**.

Hash Based Indexing

- Introduction
- Types of Hashing
 - Static Hashing
 - **Dynamic Hashing**

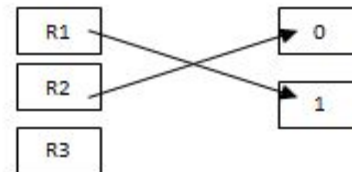
Hashing: Dynamic Hashing

- This hashing method is used to overcome the problems of static hashing – bucket overflow.
- In this method of hashing, data buckets grows or shrinks as the records increases or decreases.
- This method of hashing is also known as extendable hashing method.
- Dynamic hashing provides a mechanism in which data buckets are added and removed dynamically and on-demand.

Hashing: Dynamic Hashing

- Consider there are three records **R1**, **R2** and **R4** in the table.
- These records generate addresses 100100, 010110 and 110110 respectively.
- Dynamic hashing algorithm initially considers only part of the address – especially only first one bit to store the data.
- So it tries to load three of them at address 0 and 1.

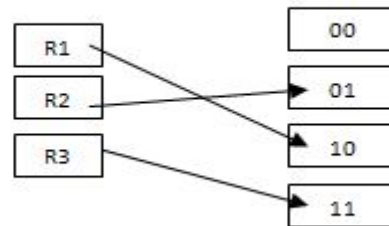
$h(R1) \rightarrow 100100$
 $h(R2) \rightarrow 010110$
 $h(R3) \rightarrow 110110$



Hashing: Dynamic Hashing

- **What will happen to R3 here?** There is no bucket space for R3.
- The **bucket has to grow dynamically** to accommodate R3.
- So it **changes the address to 2 bits rather than 1 bit**, and then it updates the existing data to have 2 bit address.
- Then it tries to accommodate R3.

$h(R1) \rightarrow 100100$
 $h(R2) \rightarrow 010110$
 $h(R3) \rightarrow 110110$



Hashing: Dynamic Hashing

- Performance does not come down as the data grows in the system.
- It simply increases the memory size to accommodate the data.
- Since it grows and shrinks with the data, memory is well utilized. There will not be any unused memory lying.
- Good for dynamic databases where data grows and shrinks frequently.

UNIT V:

- Transaction Concept: Transaction State, ACID properties, Concurrent Executions, Serializability, Recoverability, Implementation of Isolation, Testing for Serializability, lock based, time stamp based, optimistic, concurrency protocols, Deadlocks, Failure Classification, Storage, Recovery and Atomicity, Recovery algorithm.
- Introduction to Indexing Techniques: B+ Trees, operations on B+Trees, Hash Based Indexing

B-trees

- B-trees and B+ -trees are special cases of the **well-known search data structure known as a tree.**
- A tree is formed of **nodes.**
- Each node in the tree, except for a **special node called the root**, has one parent node and zero or more child nodes.
- The **root node has no parent.**
- A node that does not have any child nodes is called a **leaf node**; a non leaf node is called an internal node.
- The level of the **root node being zero.**

B-trees

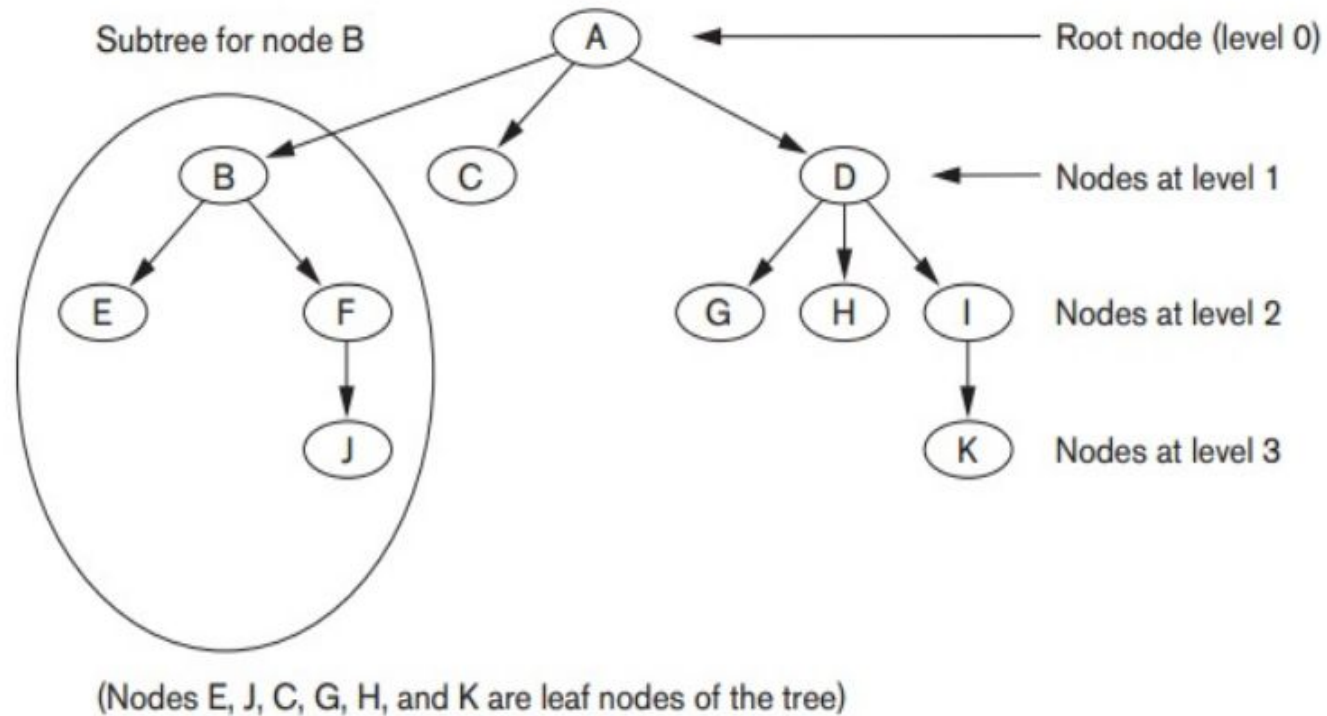


Figure 18.7

A tree data structure that shows an unbalanced tree.

B-trees

- B-Tree is a **self-balancing search tree**.
- In most of the other self-balancing search trees (like AVL and Red Black Trees), it is **assumed that everything is in main memory**.
- The main idea of using B-Trees is **to reduce the number of disk accesses**.
- B-Tree is a **Dynamic Multilevel index tree**.
- A node **can have multiple children**.

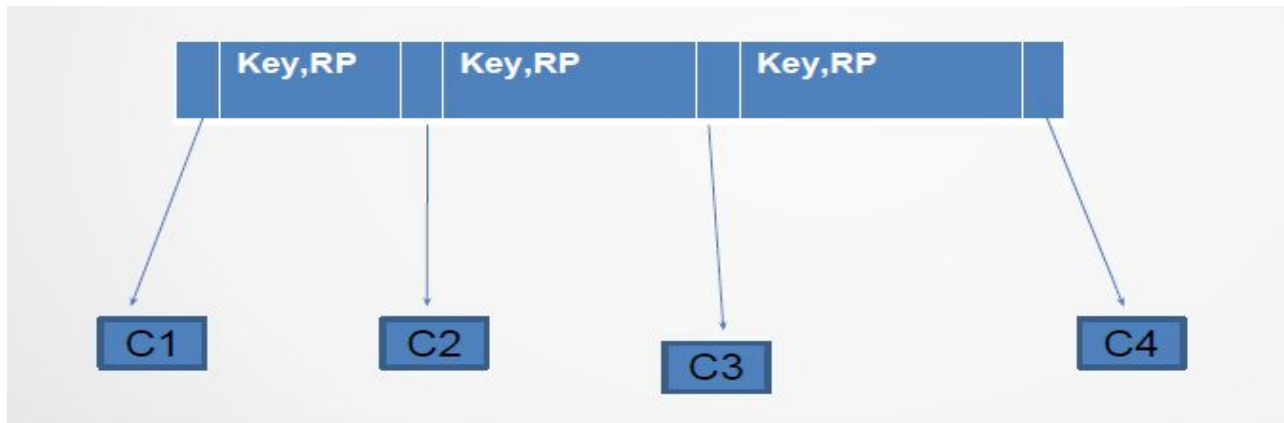
B-trees

Key Terms:

- Trees are Acyclic
- All the leaf nodes must be at same level.
- Block Pointer or Tree Pointer (Edges) points to left or right nodes.
- Data nodes contains Keys and Data pointer or Record Pointer.
- Record Pointer contains link to information stored in second memory.
- Number of keys = Number of Record Pointers.
- Block Pointer=Number of Children

B-trees

- If the order given is “p” then:
- **Max:** Block Pointers = p (Max Value)=4
- **Max:** Keys or Record Pointers = $p-1=3$
- **Min:** Block Pointers = $p/2$ (Max Value)=2
- **Max:** Keys or Record Pointers = $(p/2)-1=1$



B-trees

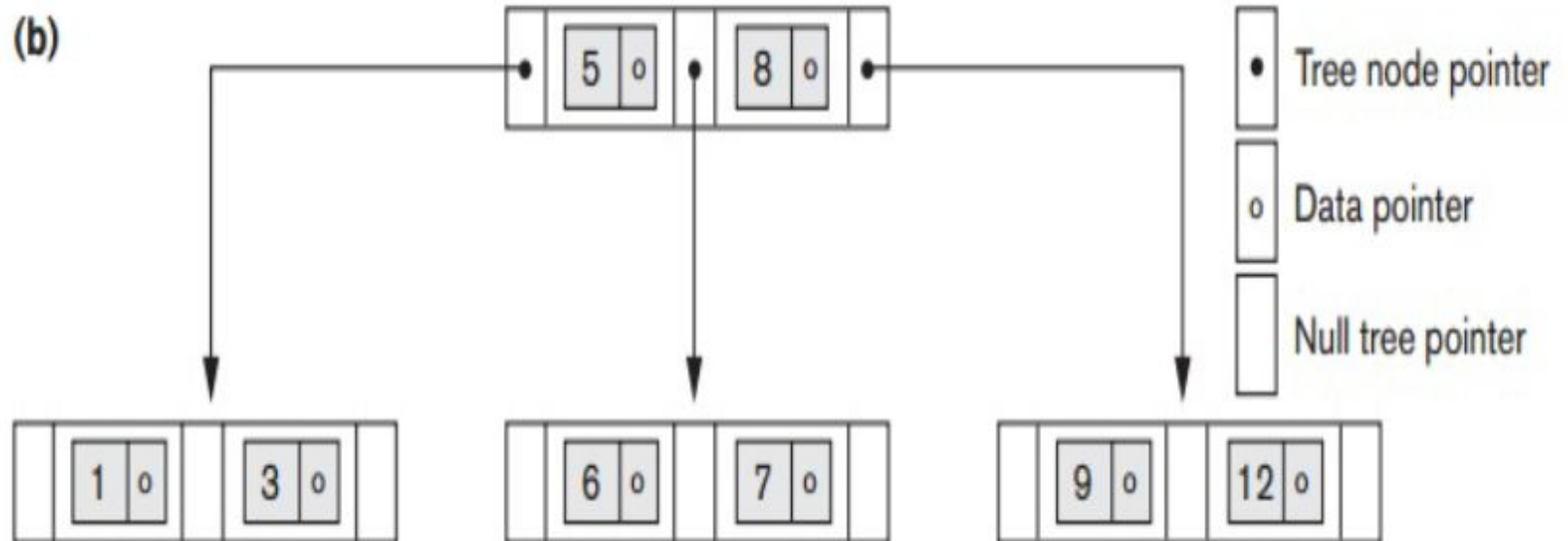
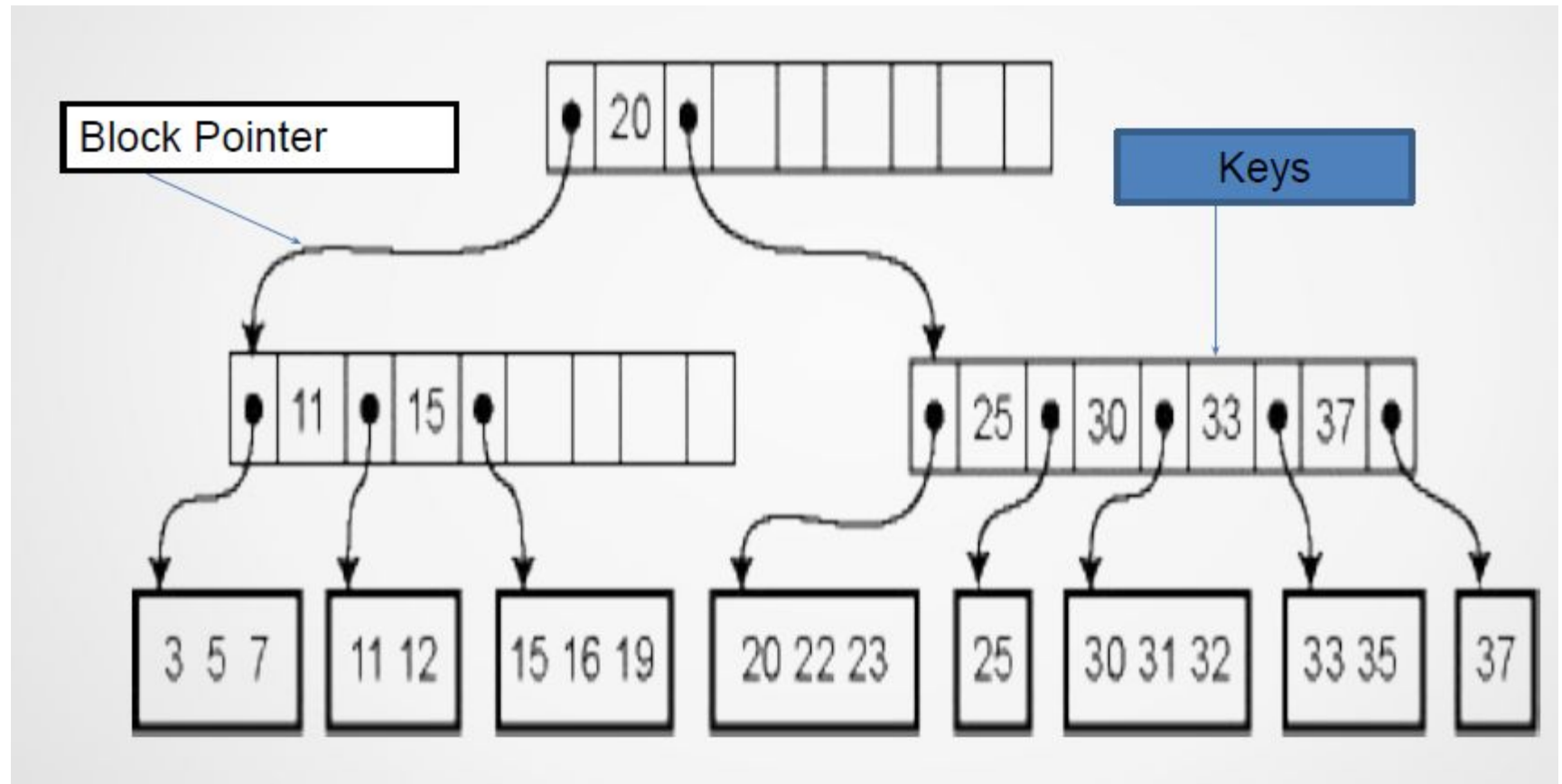


Figure 18.10

B-tree structures. (a) A node in a B-tree with $q - 1$ search values. (b) A B-tree of order $p = 3$. The values were inserted in the order 8, 5, 1, 7, 3, 12, 9, 6.

B-trees



B-trees: Example

- Insertion Example: 1,2,3,4,5,6,7,8,9,10 (* for Insertion B-Tree uses Binary Search Tree Principle)
- Maximum Degree: 4

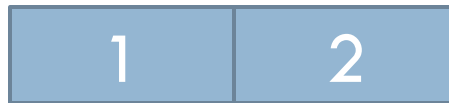
B-trees: Example

- Insertion Example: 1,2,3,4,5,6,7,8,9,10
- Maximum Degree: 4



B-trees: Example

- Insertion Example: 1,2,3,4,5,6,7,8,9,10
- Maximum Degree: 4



B-trees: Example

- Insertion Example: 1,2,3,4,5,6,7,8,9,10
- Maximum Degree: 4



B-trees: Example

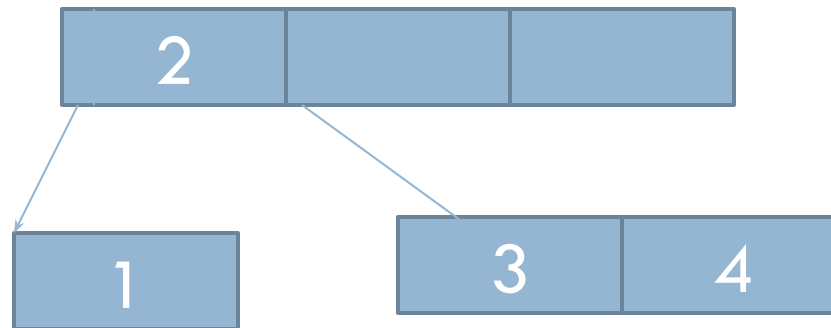
- Insertion Example: 1,2,3,4,5,6,7,8,9,10
- Maximum Degree: 4



- Order=4 i.e Max Keys= $p-1 = 4-1=3$
- Median: {1,2,3,4}= 2 (Left Order)

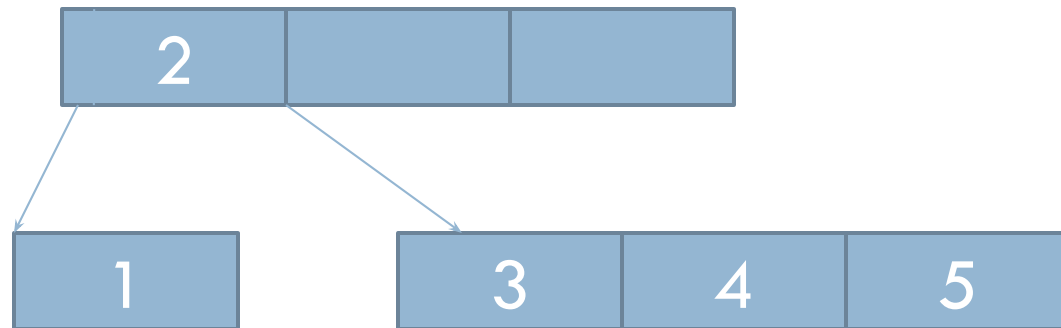
B-trees: Example

- Insertion Example: 1,2,3,4,5,6,7,8,9,10
- Maximum Degree: 4



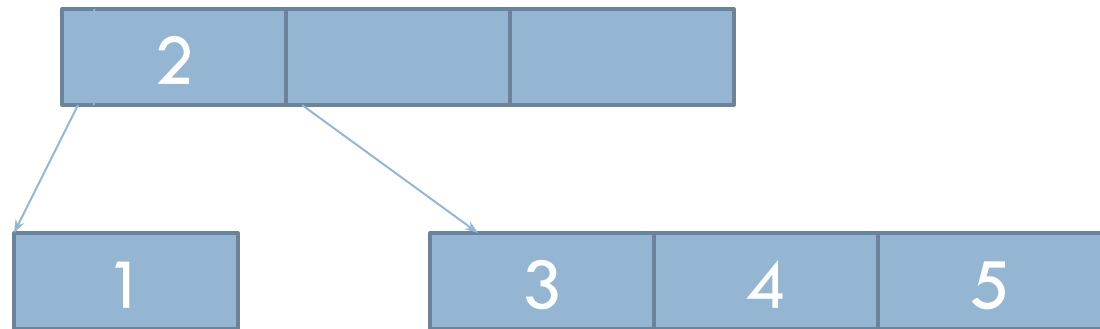
B-trees: Example

- Insertion Example: 1,2,3,4,5,6,7,8,9,10
- Maximum Degree: 4



B-trees: Example

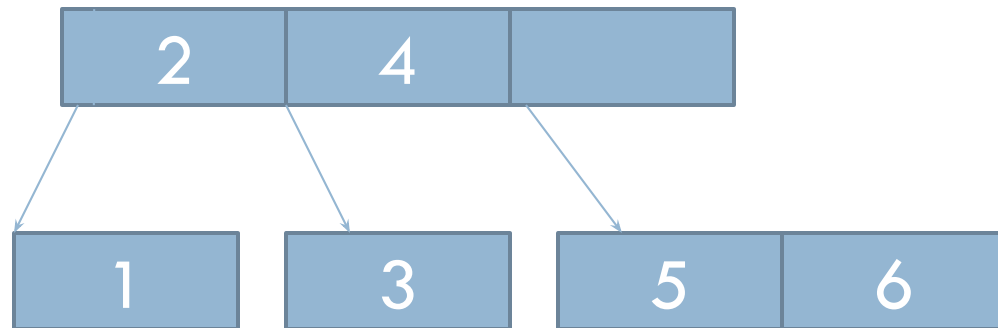
- Insertion Example: 1,2,3,4,5,6,7,8,9,10
- Maximum Degree: 4



- Median: {3,4,5,6}= 2 (Left Order)

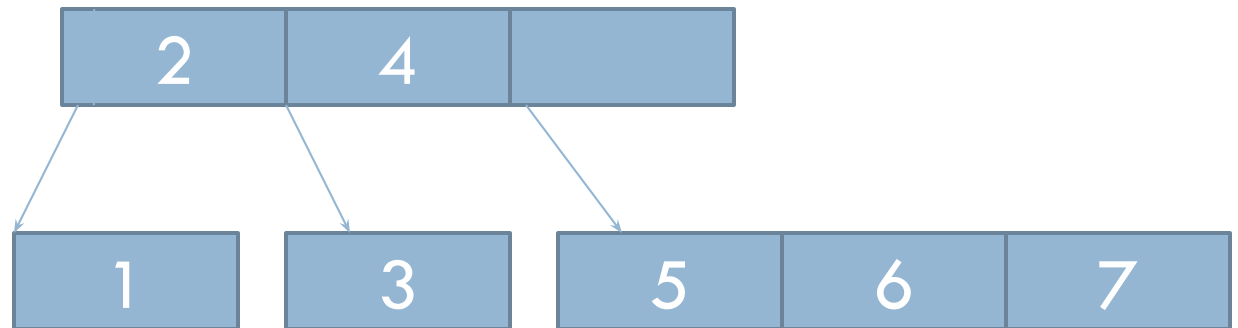
B-trees: Example

- Insertion Example: 1,2,3,4,5,6,7,8,9,10
- Maximum Degree: 4



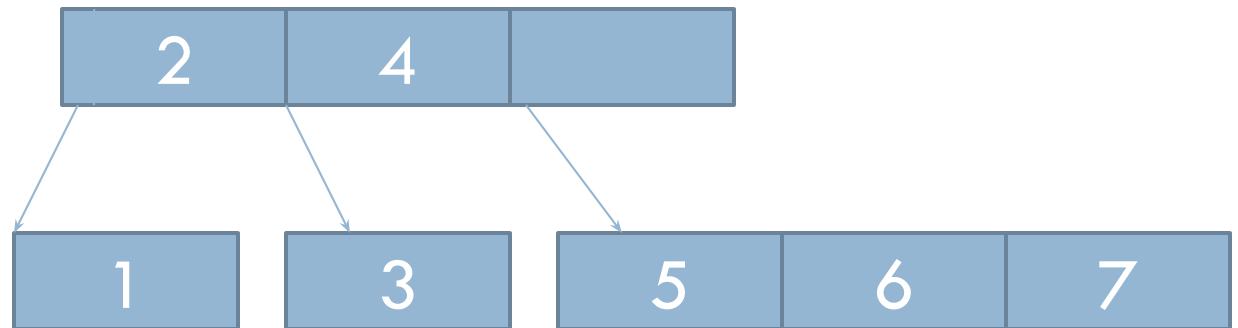
B-trees: Example

- Insertion Example: 1,2,3,4,5,6,7,8,9,10
- Maximum Degree: 4



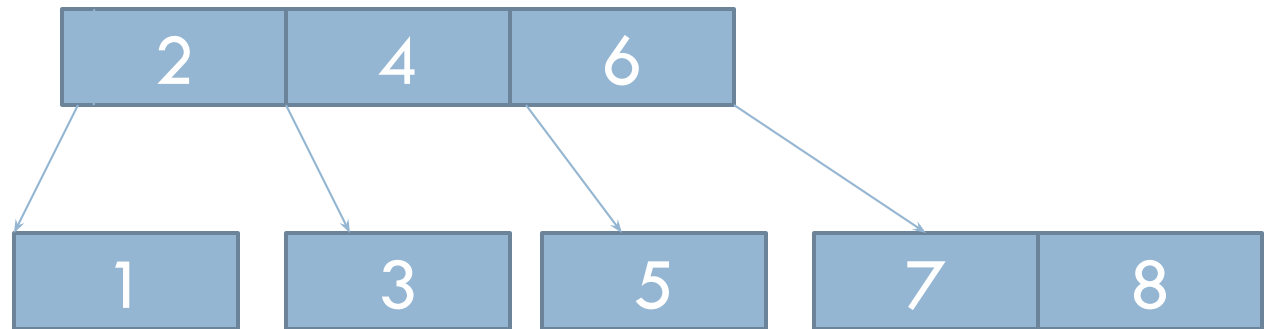
B-trees: Example

- Insertion Example: 1,2,3,4,5,6,7,8,9,10
- Maximum Degree: 4



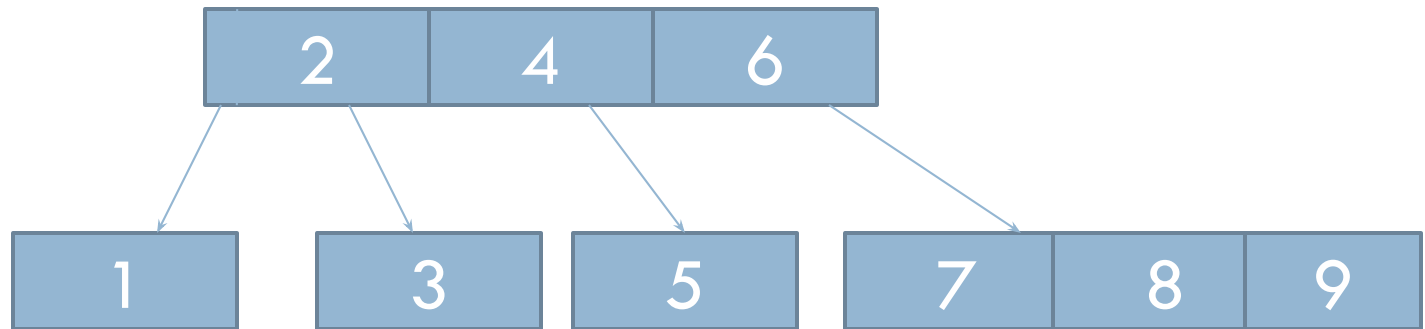
B-trees: Example

- Insertion Example: 1,2,3,4,5,6,7,8,9,10
- Maximum Degree: 4



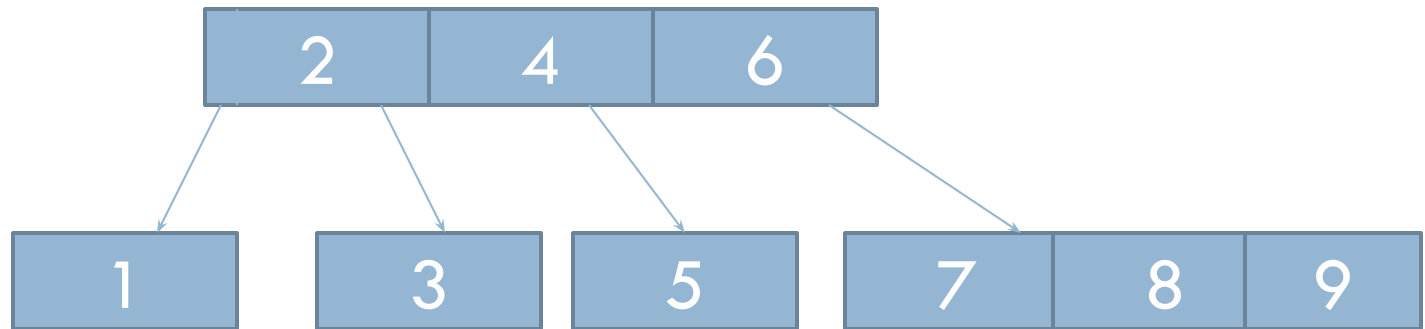
B-trees: Example

- Insertion Example: 1,2,3,4,5,6,7,8,9,10
- Maximum Degree: 4



B-trees: Example

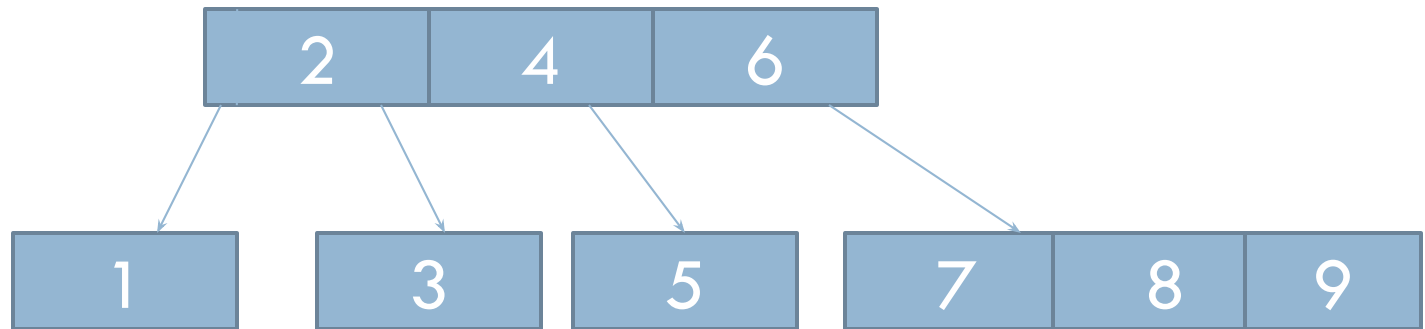
- Insertion Example: 1,2,3,4,5,6,7,8,9,10
- Maximum Degree: 4



- Median (7,8,9,10)

B-trees: Example

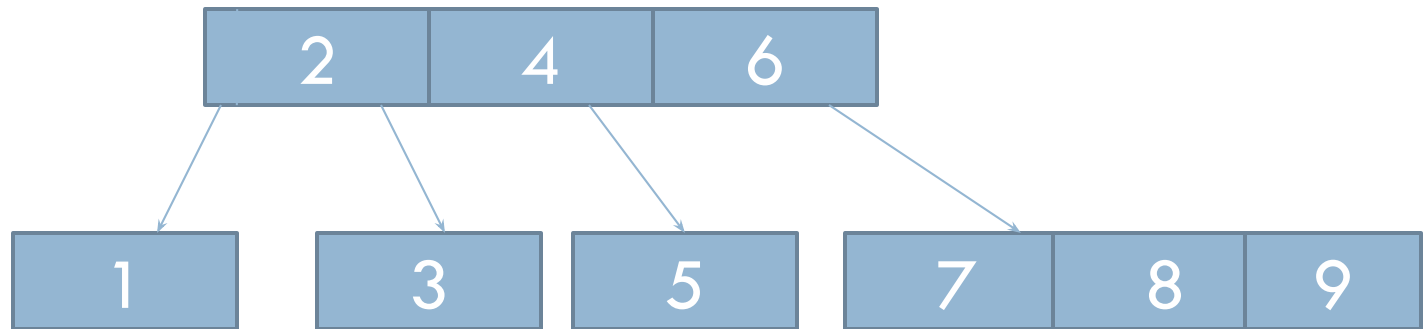
- Insertion Example: 1,2,3,4,5,6,7,8,9,10
- Maximum Degree: 4



- Median (7,8,9,10)

B-trees: Example

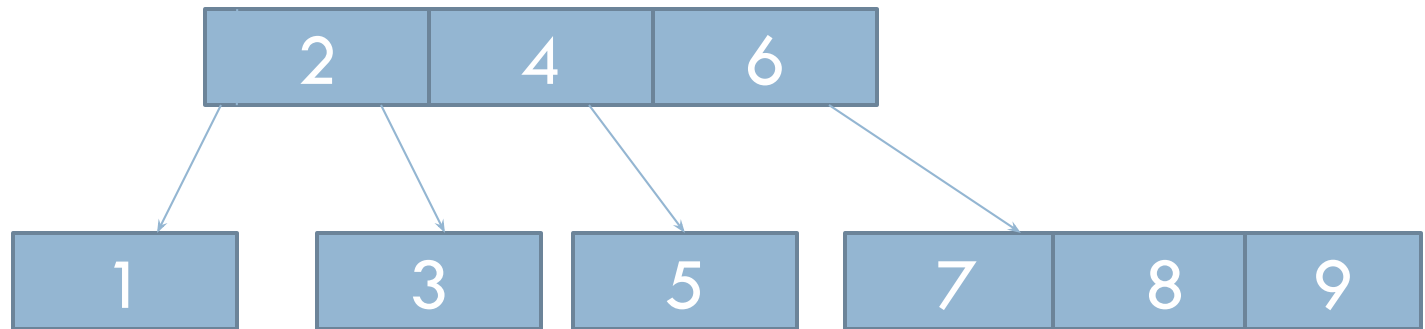
- Insertion Example: 1,2,3,4,5,6,7,8,9,10
- Maximum Degree: 4



- Median (2,4,6,8)

B-trees: Example

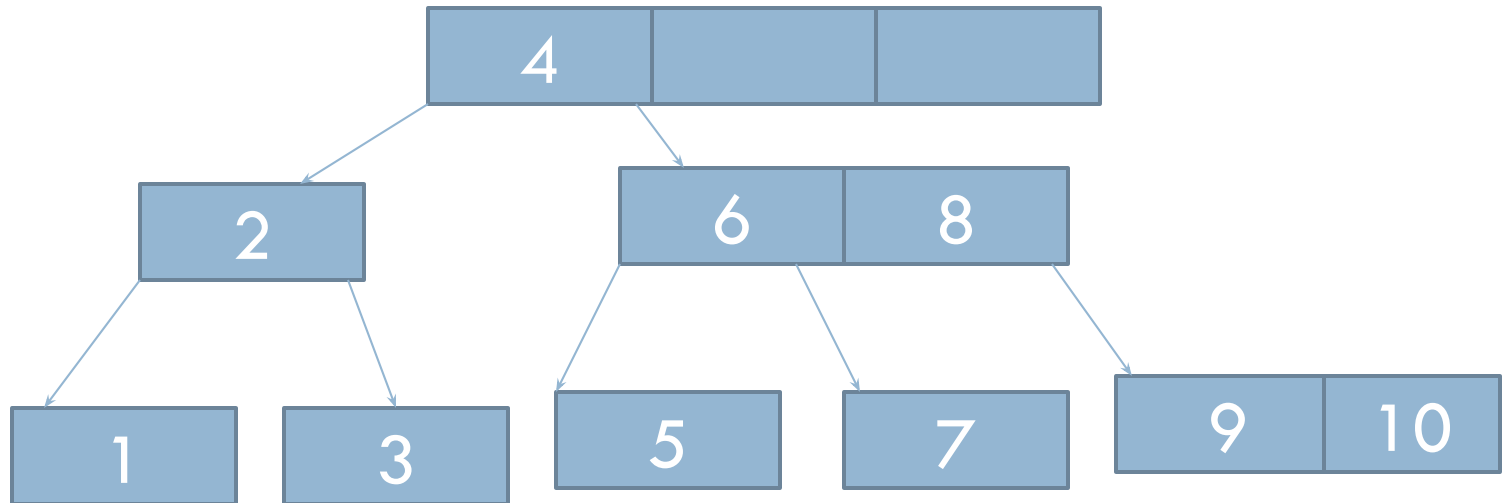
- Insertion Example: 1,2,3,4,5,6,7,8,9,10
- Maximum Degree: 4



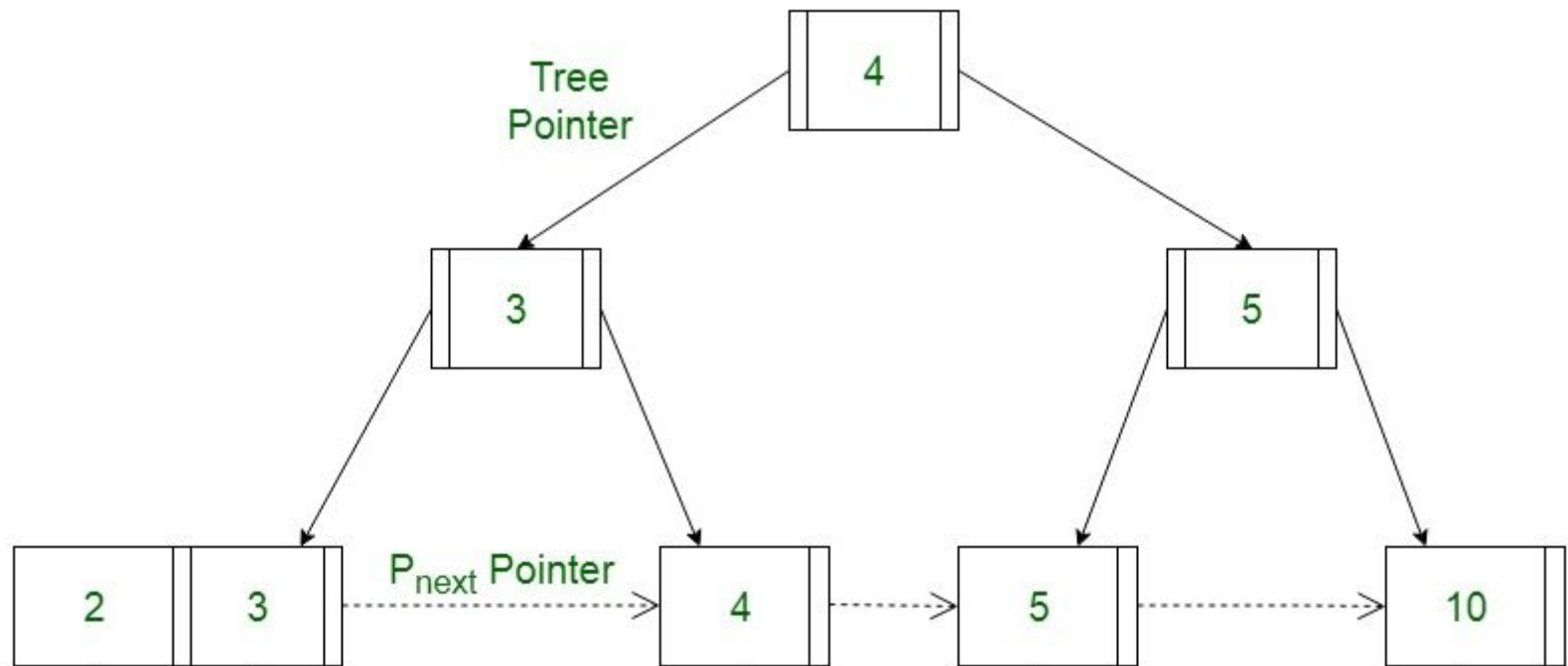
- Median (2,4,6,8)

B-trees: Example

- Insertion Example: 1,2,3,4,5,6,7,8,9,10
- Maximum Degree: 4



B+ Trees: Examples

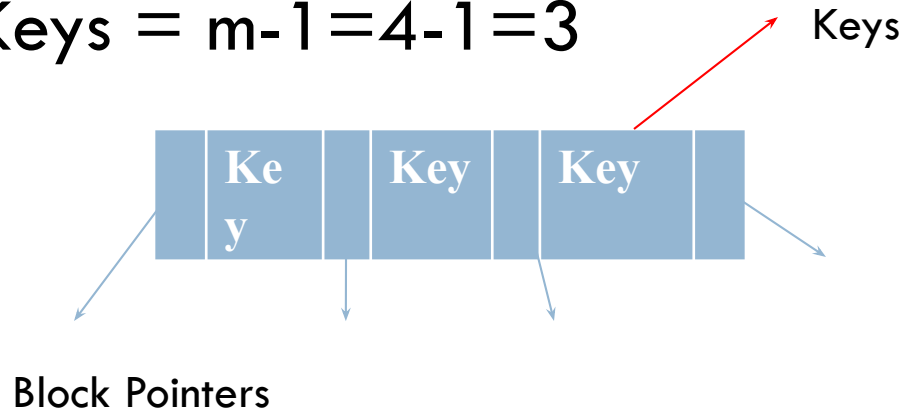


B+ Trees: Examples

- A **B+ Tree** is an advanced data structure used in database systems and file systems to maintain sorted data for fast retrieval, especially from disk.
- It is an **extended version of the B Tree**, where all actual data is stored only in the leaf nodes, while internal nodes contain only keys for navigation.
- **Components of B+ Tree:**
 - **Leaf nodes store all the key values and pointers to the actual data.**
 - **Internal nodes store only the keys** that guide searches.
 - **All leaf nodes are linked together**, supporting efficient sequential and range queries.

B+ Trees: Examples: Insertion

- **Data:** 1, 4, 7, 10, 17, 21, 31, 25, 19
- Order=4 (m)
- Max child = 4 (m)
- Max Keys = $m-1=4-1=3$



B+ Trees: Examples: Insertion

- Data: 1, 4, 7, 10, 17, 21, 31, 25, 19



B+ Trees: Examples: Insertion

- Data: 1, 4, 7, 10, 17, 21, 31, 25, 19



B+ Trees: Examples: Insertion

- Data: 1, 4, 7, 10, 17, 21, 31, 25, 19



B+ Trees: Examples: Insertion

- Data: 1, 4, 7, 10, 17, 21, 31, 25, 19



- Median = {1, 4, 7, 10}

B+ Trees: Examples: Insertion

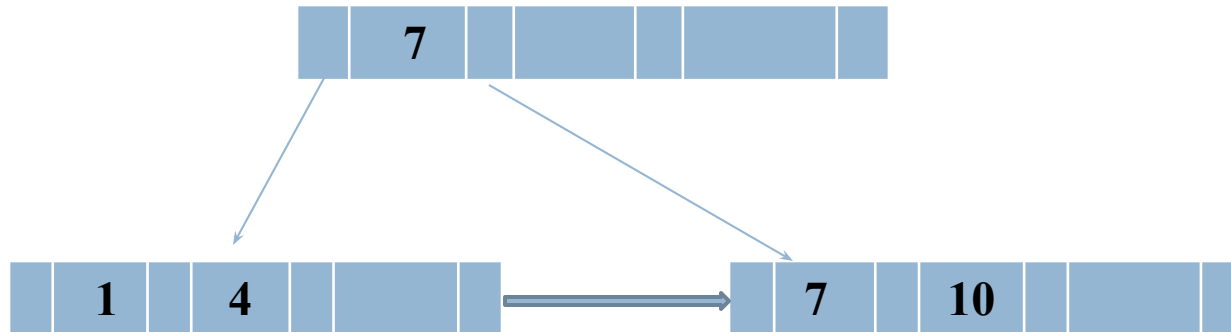
- Data: 1, 4, 7, 10, 17, 21, 31, 25, 19



- Median = {1, 4, 7, 10} = 7 (right order)

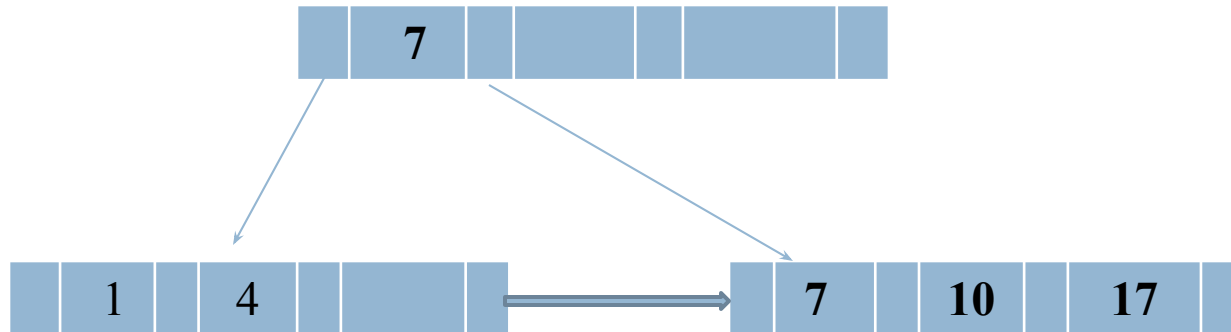
B+ Trees: Examples: Insertion

- Data: 1, 4, 7, 10, 17, 21, 31, 25, 19



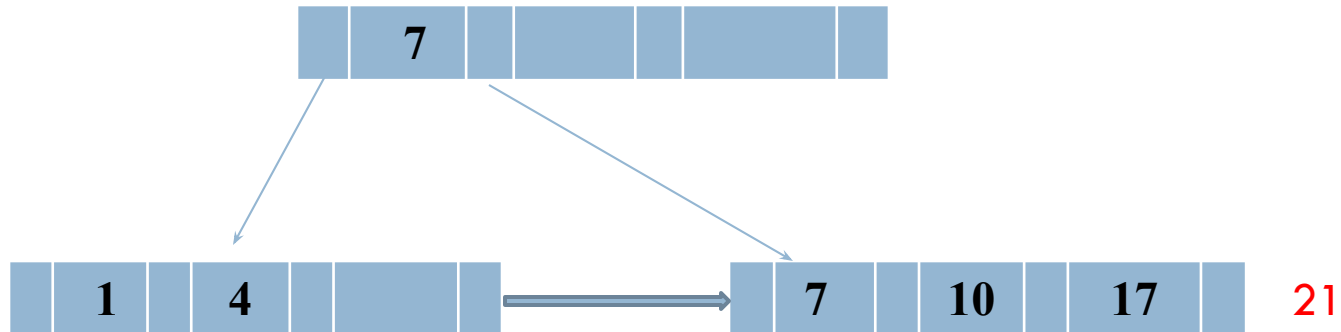
B+ Trees: Examples: Insertion

- Data: 1, 4, 7, 10, 17, 21, 31, 25, 19



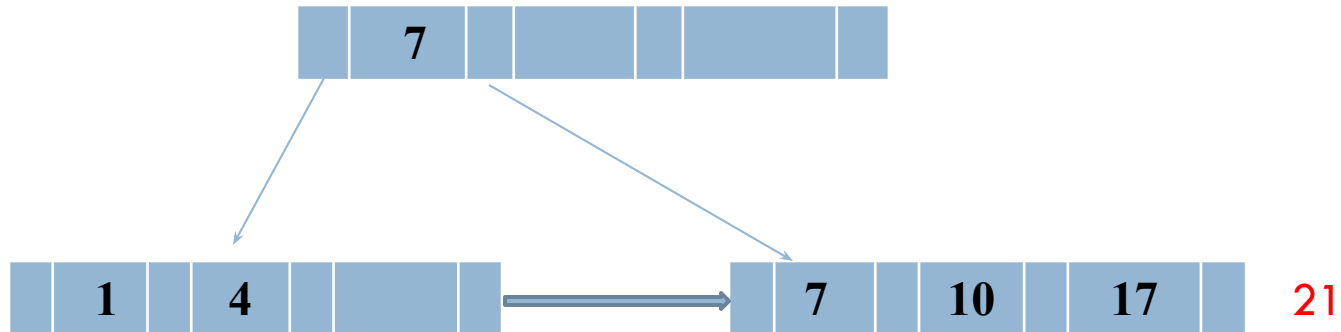
B+ Trees: Examples: Insertion

- Data: 1, 4, 7, 10, 17, 21, 31, 25, 19



B+ Trees: Examples: Insertion

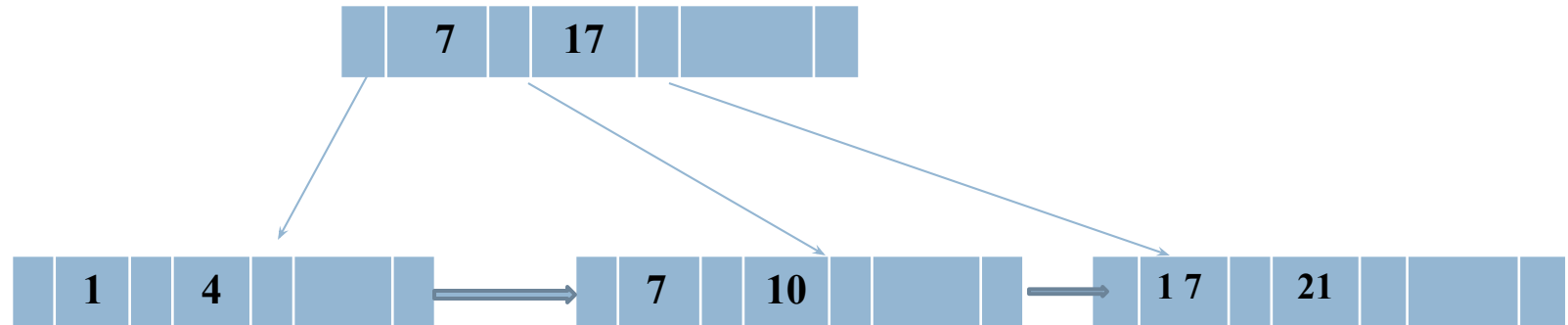
- Data: 1, 4, 7, 10, 17, 21, 31, 25, 19



- Median=(7,10,17,21)=17(right order)

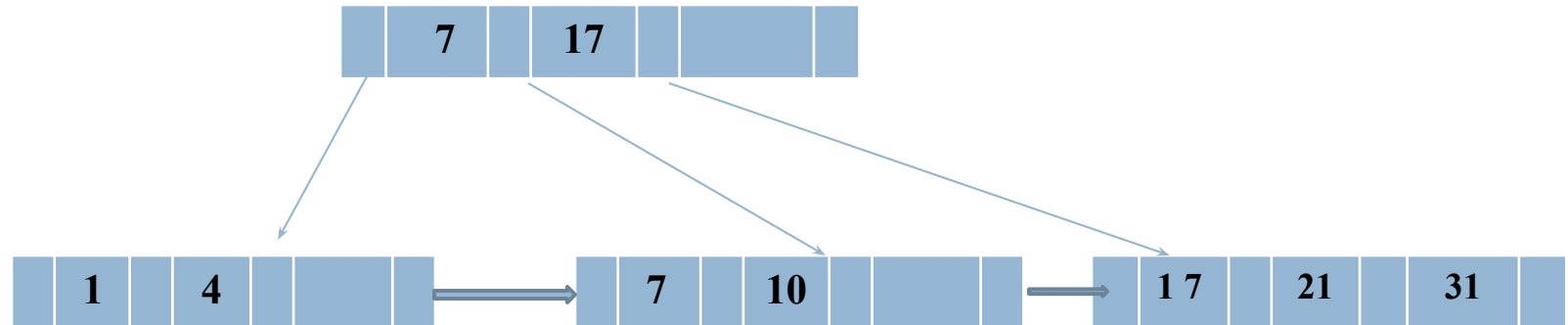
B+ Trees: Examples: Insertion

- Data: 1, 4, 7, 10, 17, 21, 31, 25, 19



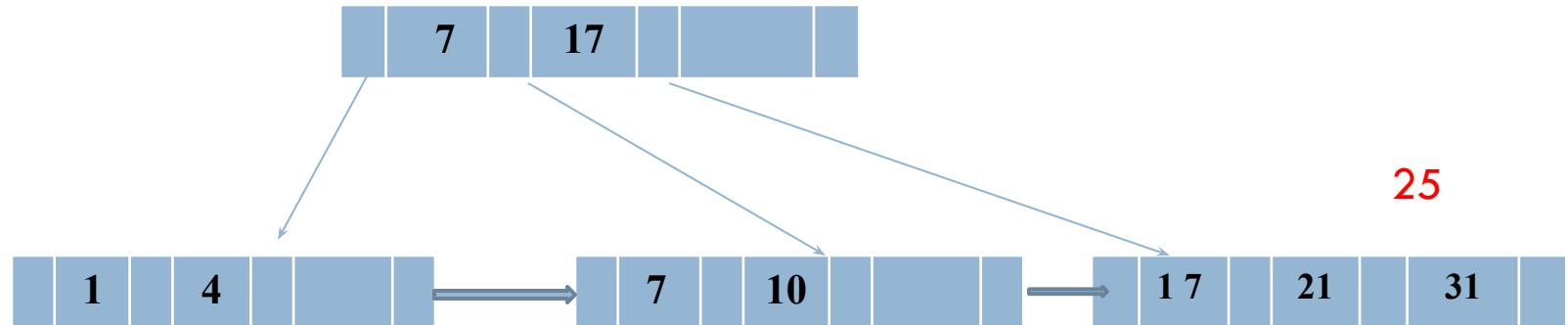
B+ Trees: Examples: Insertion

- Data: 1, 4, 7, 10, 17, 21, 31, 25, 19



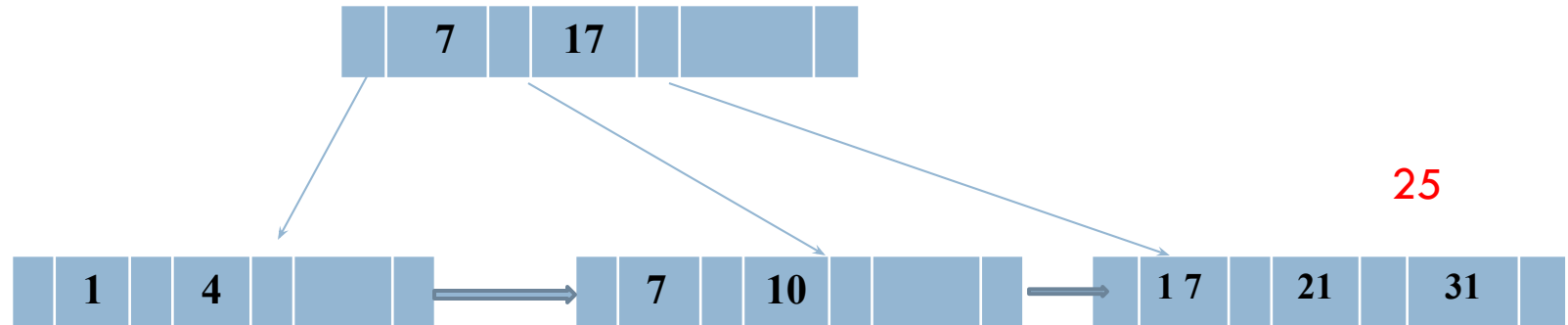
B+ Trees: Examples: Insertion

- Data: 1, 4, 7, 10, 17, 21, 31, 25, 19



B+ Trees: Examples: Insertion

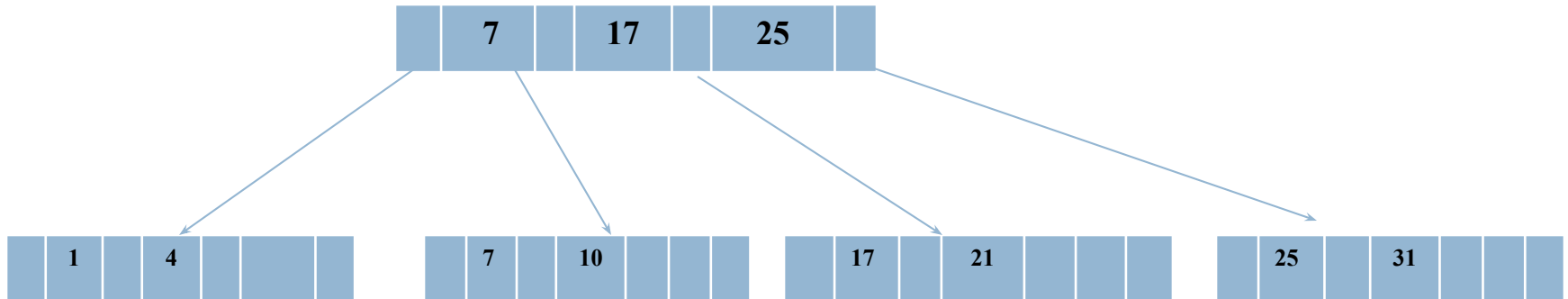
- Data: 1, 4, 7, 10, 17, 21, 31, 25, 19



- Median=(17,21,25,31)=25

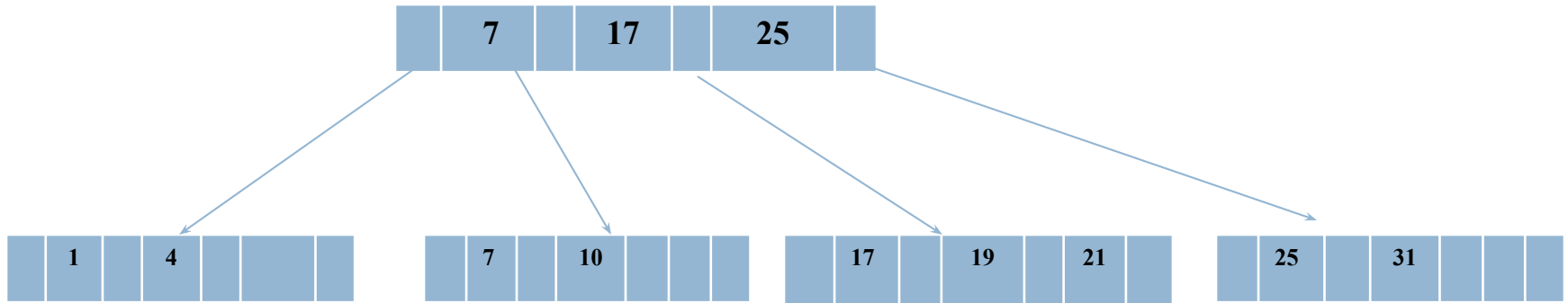
B+ Trees: Examples: Insertion

□ Data: 1, 4, 7, 10, 17, 21, 31, 25, 19



B+ Trees: Examples: Insertion

□ Data: 1, 4, 7, 10, 17, 21, 31, 25, 19



B+ Trees

Basis of Comparison	B tree	B+ tree
Pointers	All internal and leaf nodes have data pointers	Only leaf nodes have data pointers
Search	Since all keys are not available at leaf, search often takes more time.	All keys are at leaf nodes, hence search is faster and more accurate.
Redundant Keys	No duplicate of keys is maintained in the tree.	Duplicate of keys are maintained and all nodes are present at the leaf.
Insertion	Insertion takes more time and it is not predictable sometimes.	Insertion is easier and the results are always the same.
Leaf Nodes	Leaf nodes are not stored as structural linked list.	Leaf nodes are stored as structural linked list.